

Introduction to HydroServer: Architecture, Data Models & Deployment Options

Sara Alonso Vicario & Emily Dubois

Applied Scientist & UI/UX Developer, Center for Geospatial Solutions

Session 4a. Introduction to HydroServer

Tuesday, September 15, 2026

09:00 – 10:40



Utah Water Research Laboratory
UtahStateUniversity



This workshop is supported with funding from the United States government.



Photo: [The Nile River](#)

Your Trainers, Support & Development Team

Your trainers



Sara Alonso Vicario - Presenting

Applied Scientist, Center for Geospatial Solutions, Lincoln Institute

Expertise: PhD in Hydrology, continental hydrological analysis and modeling.



Emily Dubois - Supporting

UI/UX Engineer, Center for Geospatial Solutions, Lincoln Institute

Expertise: Software development, user experience (UX), and application design.

Contact Us after the Training

Contact Information:

Sara Alonso Vicario - svicario@lincolninst.edu

Emily Dubois - edubois@lincolninst.edu

Online Training Support

He will join us for about 30 minutes during our final session to answer any additional questions you may have.



Jorge Luis Sanchez-Lozano

Water Resources Engineer, Aquaveo

Expertise: PhD in Hydrology, global hydrological modeling.

He was your trainer at the last workshop in Entebbe, Uganda.

HydroServer Developers - Utah State University



Jeffery S. Horsburgh

Project Lead

Professor, Civil & Environmental Engineering, Utah State University



Ken Lippold

Software Developer

Research Engineer, Utah Water Research Lab



Daniel L. Slaugh

Software Developer

Software Engineer, Utah Water Research Lab



Maurier Ramirez

Software Developer

Software Engineer, Utah Water Research Lab



Utah Water Research Laboratory
UtahStateUniversity

Contact Developer's Team

Contact Information:

Jeff Horsburgh - jeff.horsburgh@usu.edu

Contact Us

Have questions about our software or development? We'd love to hear from you.

Email Us

The HydroServer software is under development at the Utah Water Research Laboratory at Utah State University through support from the National Oceanic and Atmospheric Administration's (NOAA) Cooperative Institute for Research to Operations in Hydrology (CIROH). Please contact Jeff Horsburgh at jeff.horsburgh@usu.edu.

[Send an email](mailto:jeff.horsburgh@usu.edu)



Connect With Us on GitHub

All of our software development activities are open source and available on GitHub. You can submit bugs or issues and follow our software development activities on our repository.

[Go to GitHub](#)



Training Participant Introductions: A Quick Recap

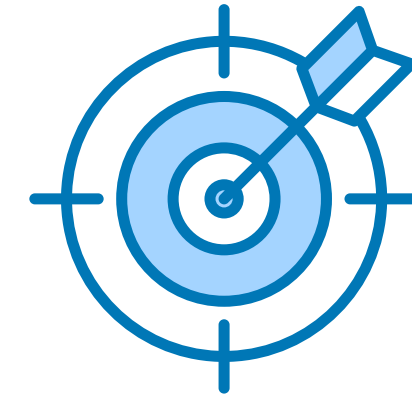
Selam!
simé...

Jambo!
jina langu ni

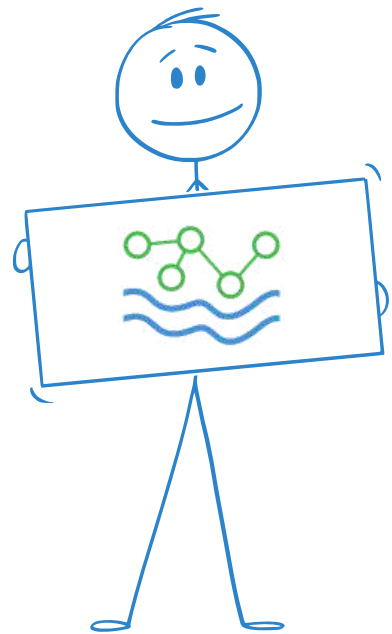


HydroServer Training

HydroServer Training Objective



This training is designed to provide participants with the practical skills needed to manage, ingest, visualize, quality-control, query, and automate hydrological data workflows using HydroServer.



HydroServer Training - Four Sessions

Session 4A – Introduction to HydroServer

- HydroServer overview, main components, and deployment options
- **Interactive exercise** on the HydroServer data model and structure

Session 5A – Getting Started with HydroServer

- Create and configure your HydroServer resources
- **Exercise 1:** Load historical hydrological data

Session 6A – Automated Data Ingestion & Quality Control

- **Exercise 2:** Loading Real-Time Sensor Data into HydroServer using the Streaming Data Loader.
- Visualize data and perform quality control

Session 7A – Querying & Advanced Automation

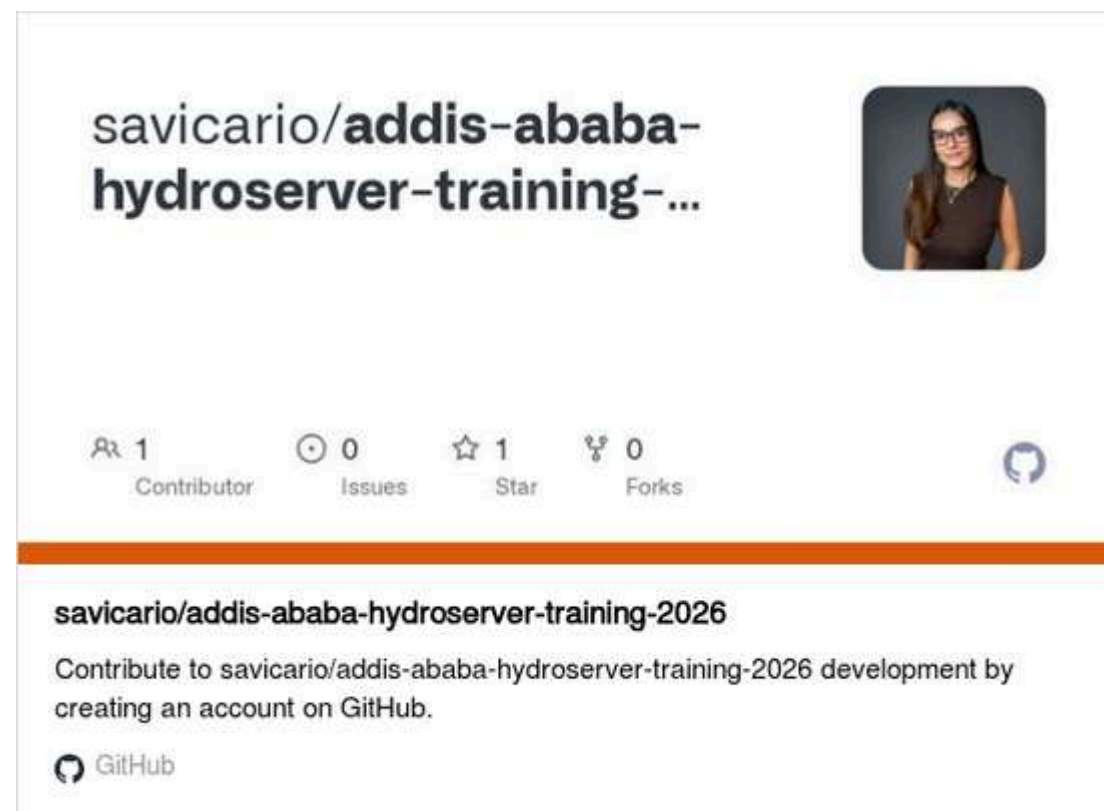
- **Exercise 3:** Query and manage HydroServer data using Python and hydroserverpy
- **Exercise 4:** Building a Custom ETL Pipeline for Automated Data Ingestion
- **Exercise 5:** Automating Data Ingestion with HydroServer ETL Tasks



HydroServer Training Repository



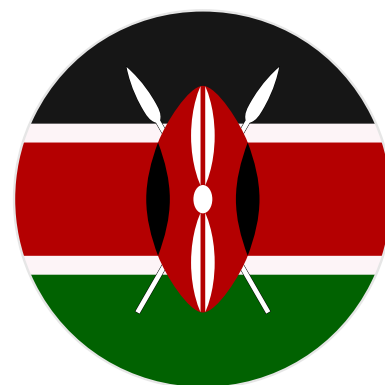
This is the link to the public repository containing the presentations and exercises for all four session



Addis Ababa, Ethiopia
September 15, 2026

HydroServer Training Resources

- **README:** Workshop introduction and overview of each session.
- **Presentations:** Canva link to the presentations for all four sessions.
- **Exercises:** Separate links to the exercises for each country.



The exercises are expected to be completed **individually**, with each participant working on their **own computer**.



The first exercise is specific to each country, and you will work with your own sample dataset. The remaining exercises are the same for all countries.

HydroServer Training Requirements



Google account to access the exercises through Google Colab.

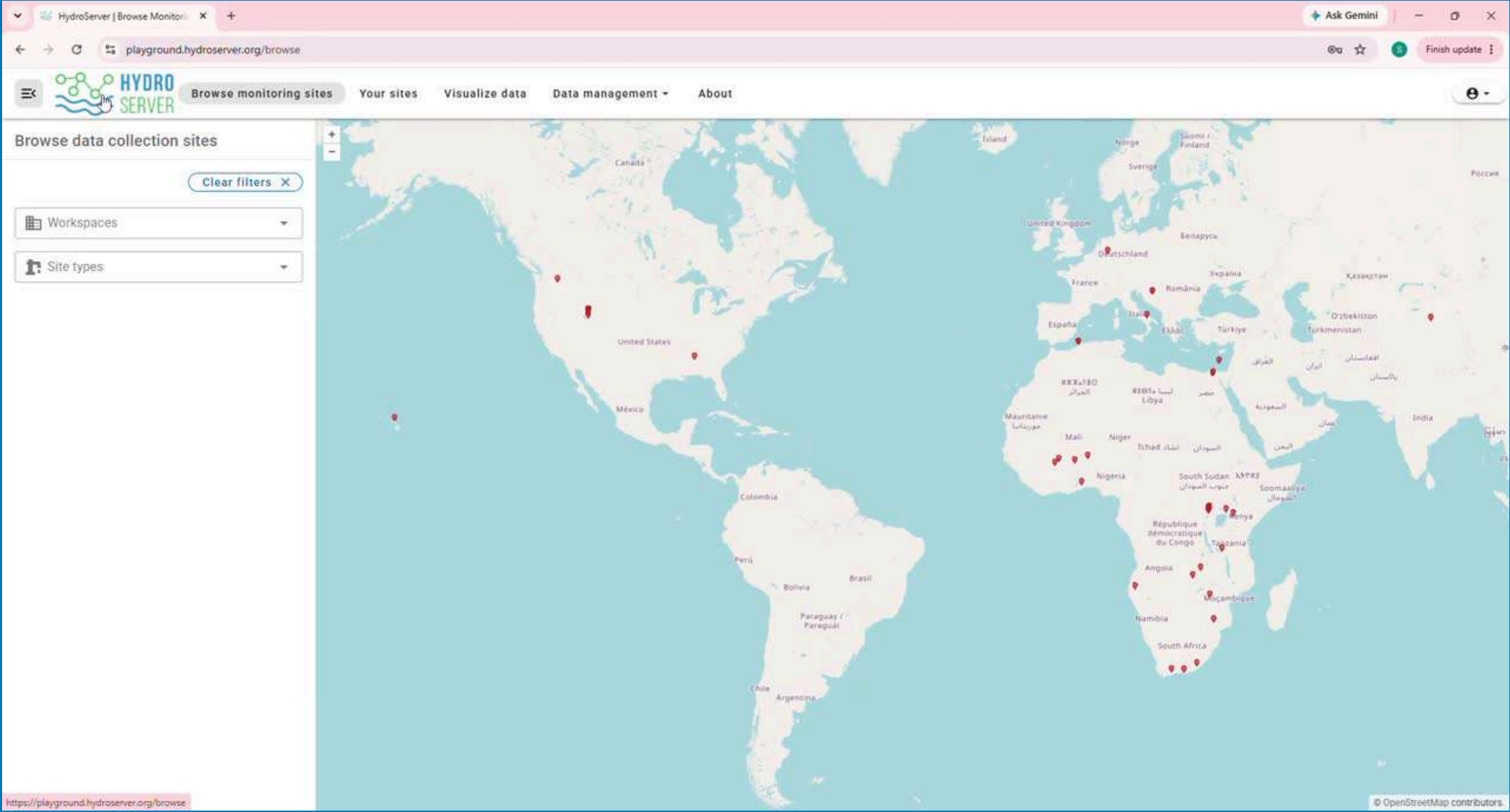


HydroServer account - if you don't have one yet, we'll walk you through the steps in the next session!



What it is HydroServer?

HydroServer is an **open-source** platform for day-to-day collection, managing, publishing, storing, and sharing environmental time series data from fixed-location monitoring systems



The screenshot shows the HydroServer playground.browse interface. On the left, there's a sidebar with 'Browse data collection sites' and filters for 'Workspaces' and 'Site types'. The main area displays a world map with numerous red dots representing data collection sites. The URL in the browser is 'playground.hydroserver.org/browse'.



HydroServer

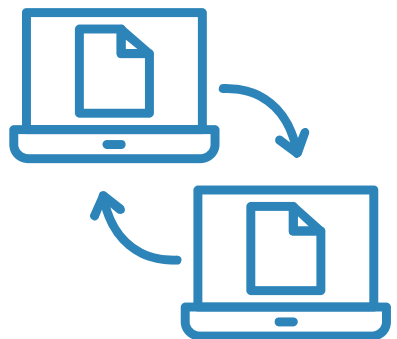
HydroServer: A comprehensive platform for water data management. Collect and manage your operational hydrologic data

 hydroserver.org



HydroServer was born out of some very practical challenges that organizations face when managing sensor data on a day-to-day basis.

- Operational and scientific use of sensor data requires **effective data management software**
- Operators of sensor networks **struggle with day-to-day data management and sharing**
- **Few reliable, modern, and generic software options** are available
 - **Some commercial options are good, but expensive**
 - **Many are tied to a specific sensor/datalogger manufacturer**



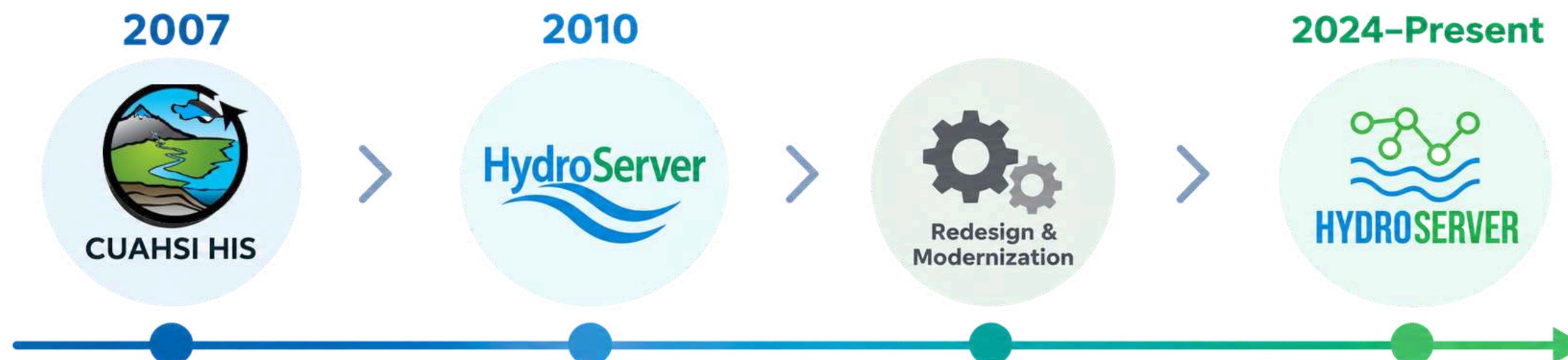
How HydroServer started?

2007 - First developed as "HIS Server" within the CUAHSI Hydrologic Information System (HIS)

2010 - HydroServer evolved into a platform for storing, managing, and publishing hydrologic observations

Later - The original technology became a legacy system, creating the need for redesign and modernization

Today - HydroServer 2 is a modern, open-source platform for managing environmental observations

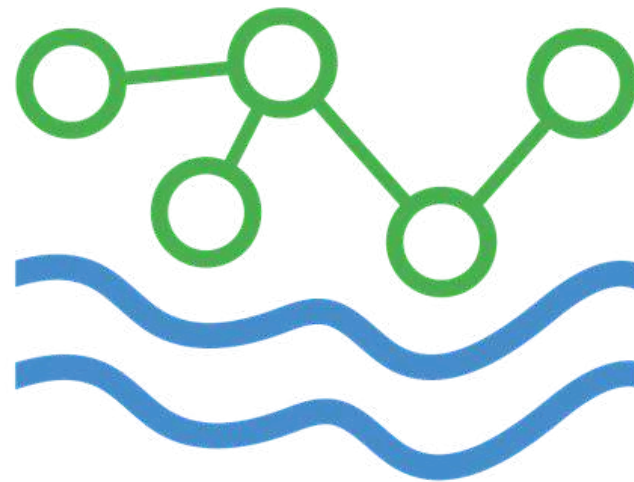


Explore HydroServer Publications

You can learn more about the technical details of HydroServer by reading the two publications linked here.



HydroServer 1



HydroServer 2



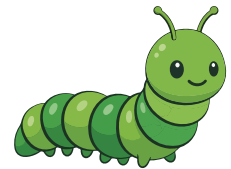
HydroServer: Free, Open & Community-Driven



HydroServer is continuously evolving



Open-source and actively developed



Users can report bugs and request new features on GitHub



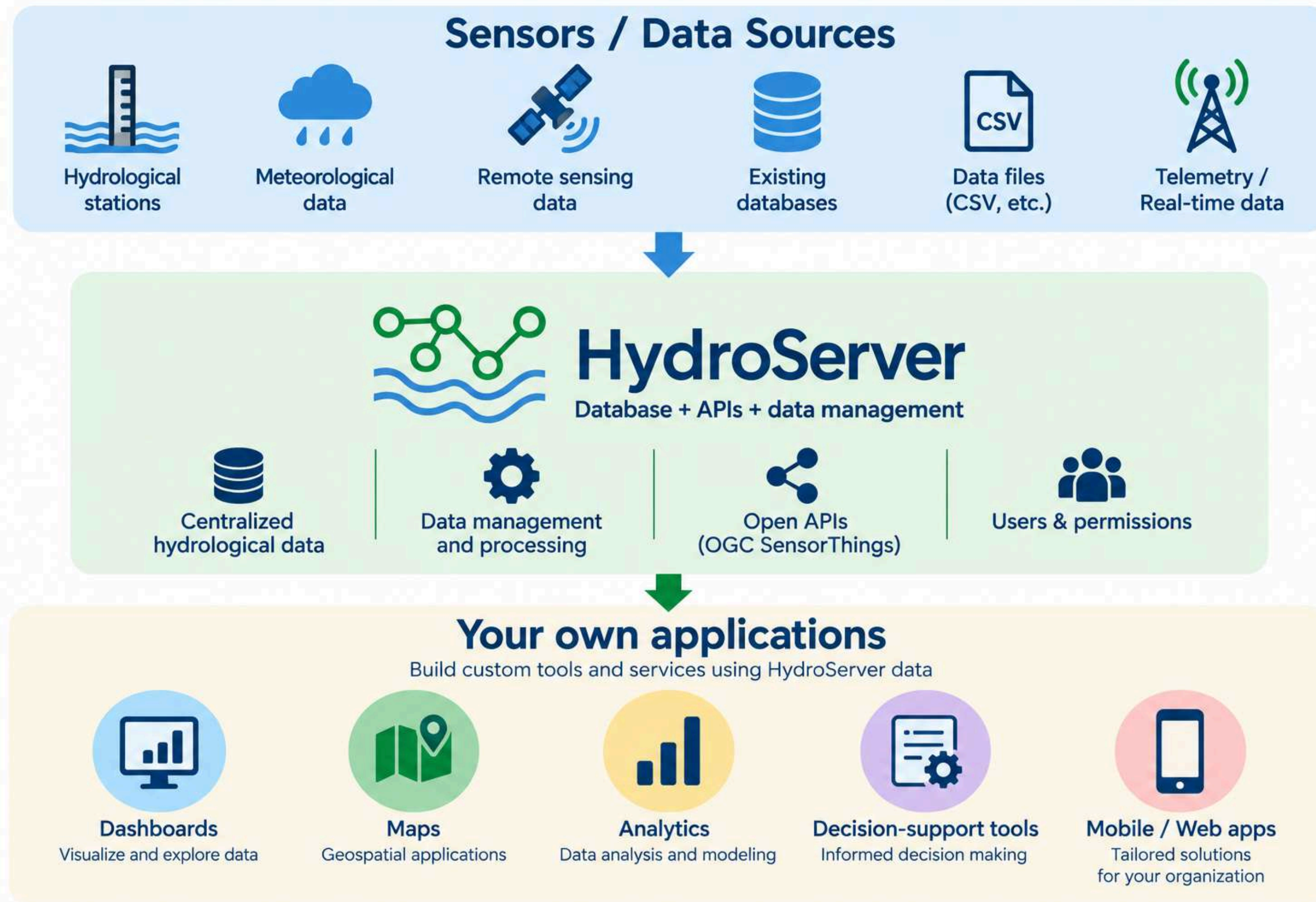
Developers can contribute code or extend HydroServer with their own tools and applications



HydroServer is **Free** to use! 



Building on HydroServer



HydroServer Open Resources

As we mentioned, HydroServer is open-source, so all of the source code is publicly available. They also provide a Playground where you can experiment with HydroServer and explore its features, as well as extensive documentation. You can access all of these resources here.



[HydroServer Github Repository](#)

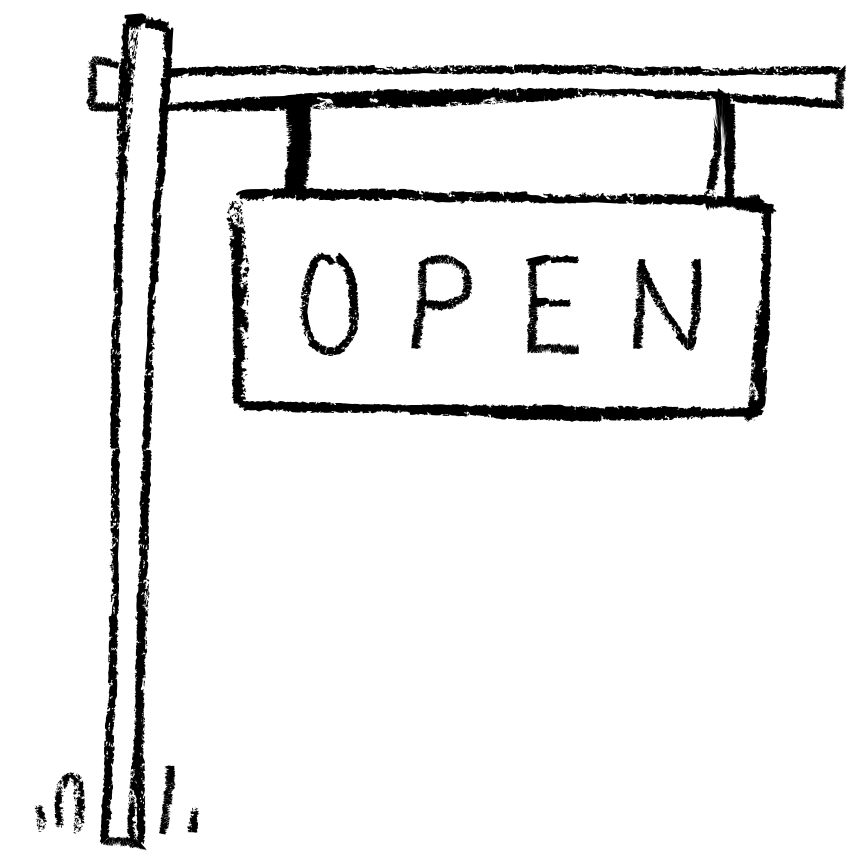


[HydroServer Demo/Playground](#)



HYDRO
SERVER

[HydroServer Documentation](#)



HydroServer & WMO



WMO has supported the development and deployment of HydroServer



Designed for interoperability with WMO hydrological data systems and standards



HydroServer has been deployed through WMO-supported projects



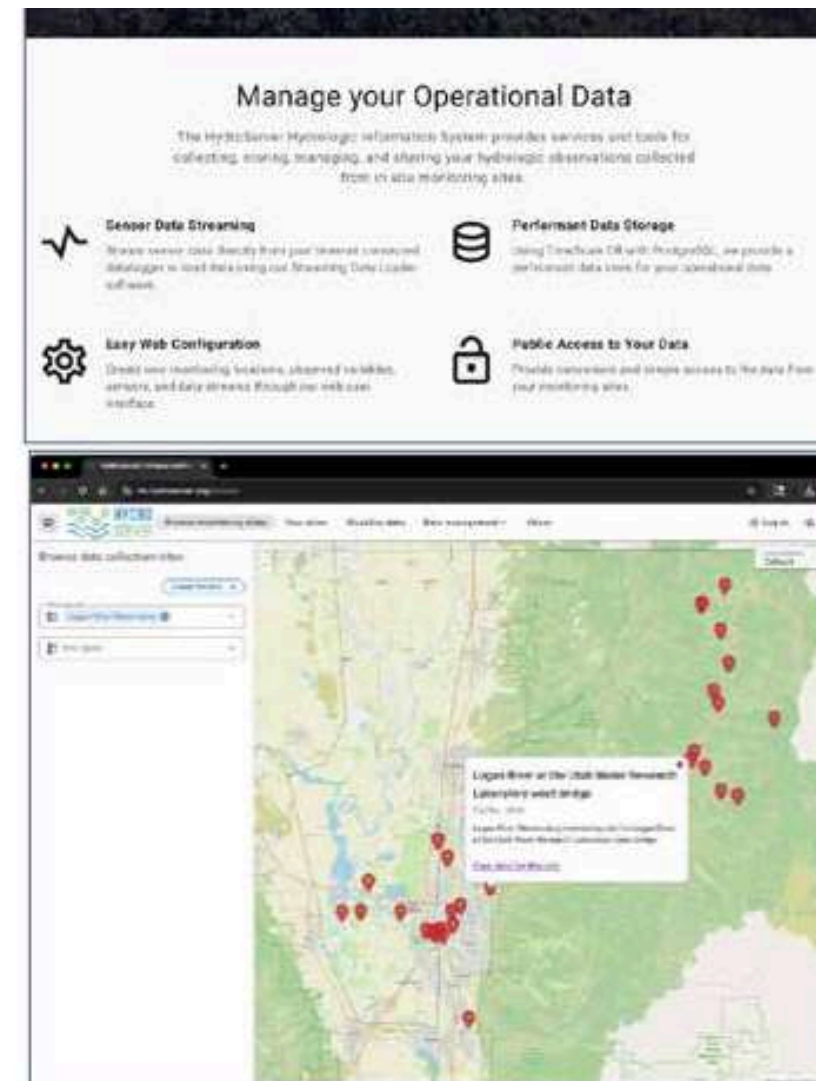
Helps connect national hydrological data → regional and global data-sharing systems

If you would like to learn more about how HydroServer and WMO are working together on HydroServer implementation projects, you can read the document linked here.

Modernized Standards and Tools for Sharing and Integrating Real-Time Hydrologic Observations Data

What we did

We (1) published three papers: one describing the HydroServer software, including a synthesis paper of existing HIS technologies and systems, one describing HydroServer's architecture, and one describing HydroServer's web services; (2) completed HydroServer software development and hosted it in open-source code repositories, (3) developed deployment scripts to automate the HydroServer software to commercial cloud computing platforms; (4) deployed and maintained instances of the HydroServer software stack to manage data for the Logan River Observatory (LRO), Utah Division of Water Rights, and multiple developing countries in Africa in collaboration with the World Meteorological Organization; and (5) trained multiple students on the technical skills for sensor data management and operation of Hydrologic Information Systems.

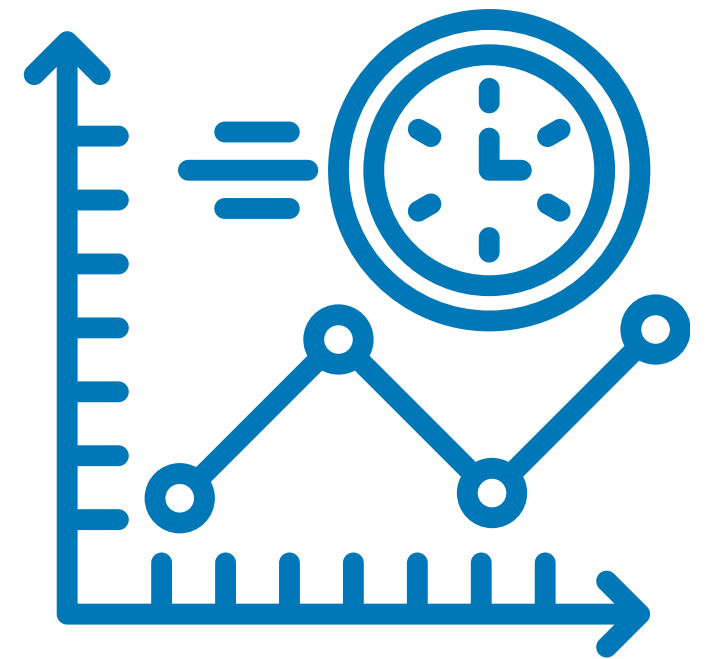
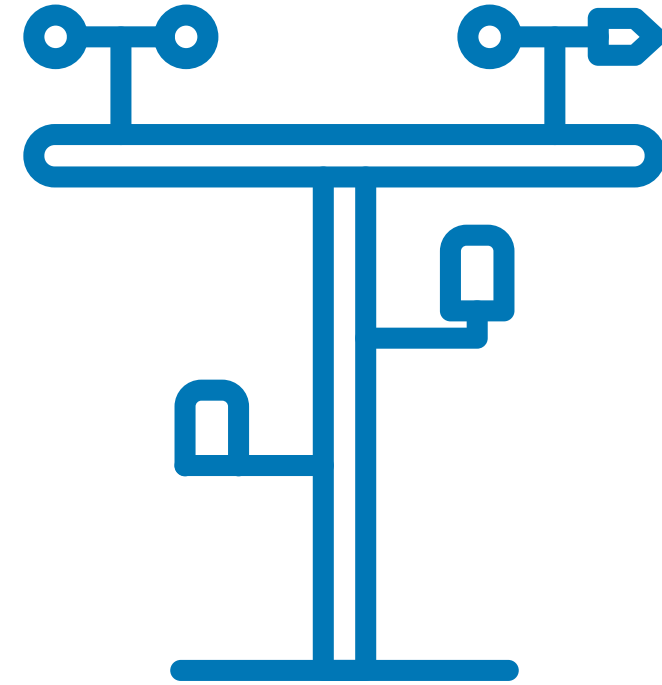


HydroServer HIS website.

[**Access the Document**](#)

What Data Can HydroServer Manage?

- Broadly - sensor time series data
- Fixed-point monitoring stations
 - Streamflow gages
 - Water Quality monitoring sites
 - Climate Stations
 - Soil sensors
 - ...
- Derived time series - does not have to come from a sensor directly
- There is nothing really “Hydro” about it - any time series can be supported



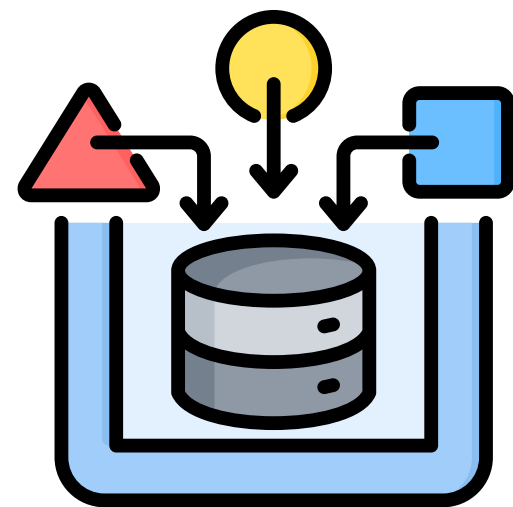
HydroServer Data Ingestion Options - Overview

HydroServer provides several options for ingesting data.

The main difference between these approaches is who is responsible for getting the data into HydroServer.

The source can push data directly (i.e., data logger), the Streaming Data Loader can monitor local files, an external system can run the ETL workflow, or HydroServer itself can manage the ETL process

- **Direct API** → Source pushes data to HydroServer
- **Streaming Data Loader** → Monitors local files
- **Externally Managed ETL** → Your system retrieves and loads the data
- **HydroServer-Managed ETL** → HydroServer retrieves and loads the data



HydroServer Data Ingestion Options - Differences



Direct API Loading

Data are **pushed directly** to HydroServer's API

External script/device (i.e., datalogger)

Training Exercise Number

2



Externally Managed ETL*

Data are **retrieved, transformed, and loaded**

Your external ETL workflow

4



HydroServer-Managed ETL

Data are **retrieved, transformed, and loaded**

HydroServer's orchestration system

5

*ETL: Extract, Transform, Load

HydroServer Data Ingestion Options - Differences

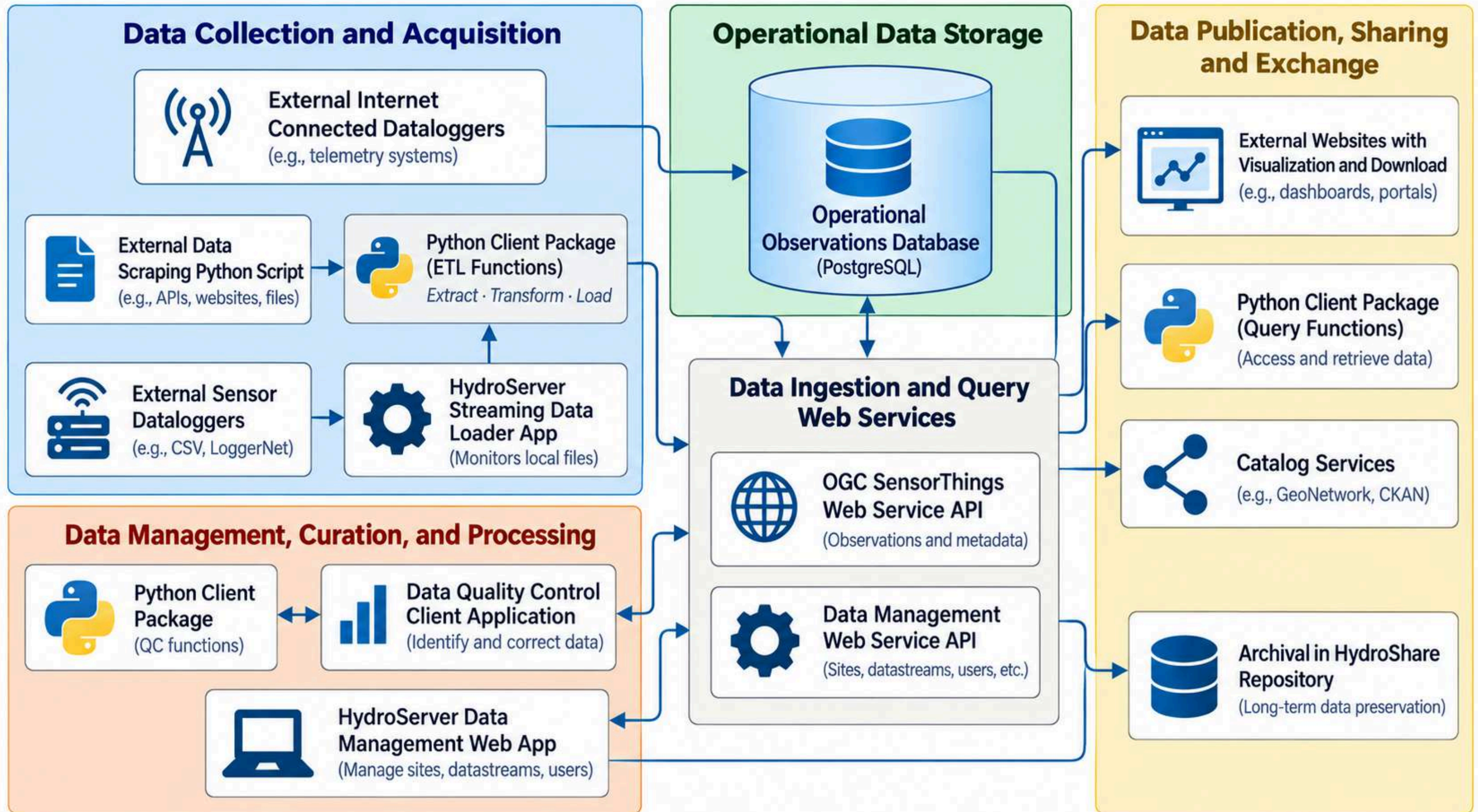
	Externally Managed ETL	HydroServer-Managed ETL
Who runs it?	You / an external system	HydroServer
Where does it run?	Outside HydroServer	Within HydroServer's job orchestration system
Who schedules it?	You	HydroServer
ETL logic	Defined in your Python workflow	Configured as a HydroServer ETL task
Best for	More custom/flexible workflows	Easier built-in automation

HydroServer Architecture



HydroServer
Architecture is the set of
components that work
together to **ingest, store,**
manage, access, and use
environmental time-series
data.

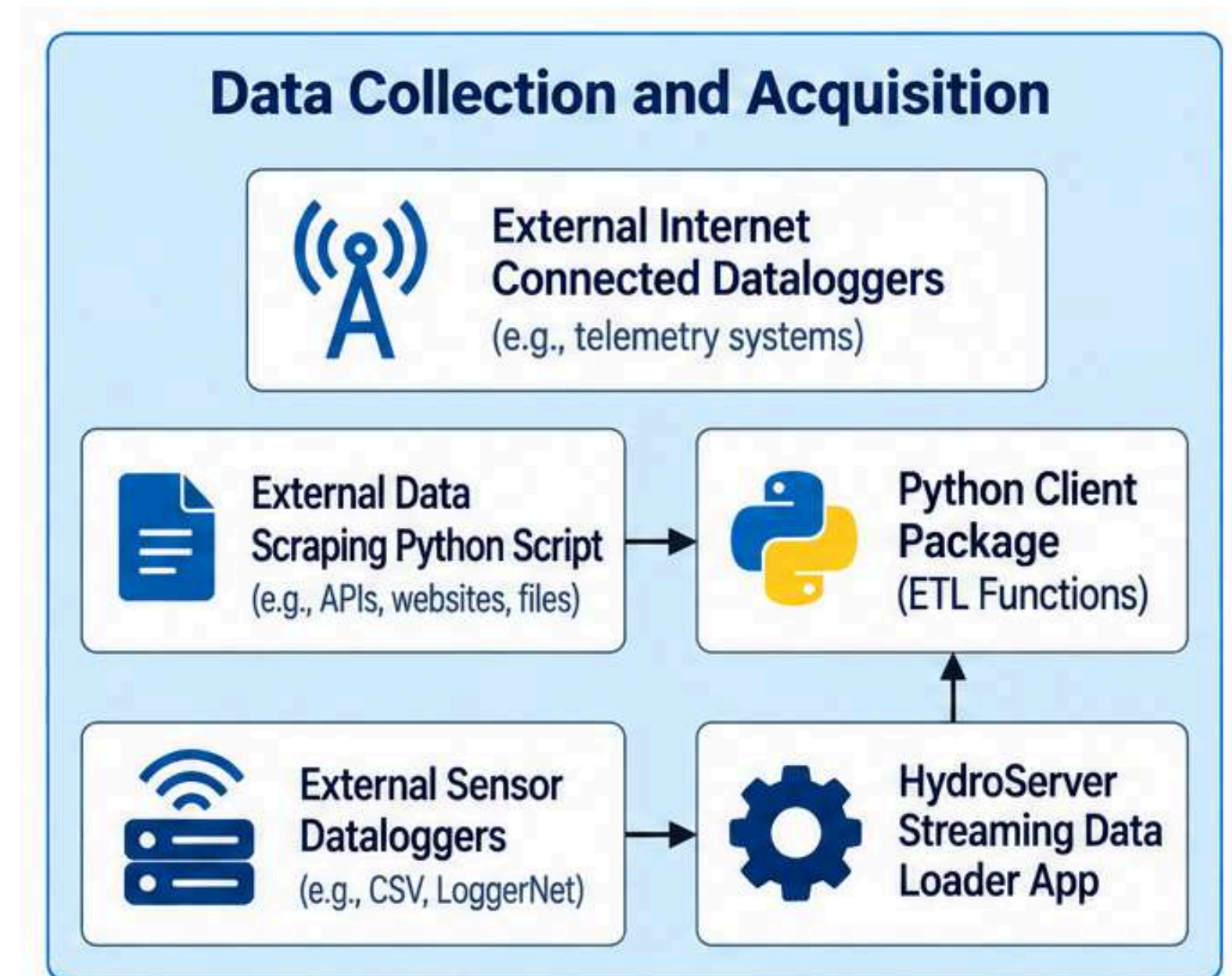
Architecture of HydroServer



Overview of HydroServer's Main Architecture Components

Data Collection and Acquisition

- Flexible and interoperable mechanisms for ingesting environmental data **without replacing existing field data collection software**
- Multiple ingestion pathways
 - CSV files
 - Custom scripts (Extract, Transform, Load) workflows
 - Internet-connected data loggers
- Supports automated data ingestion through scheduling and orchestration tools



Operational Data Storage and Web Services

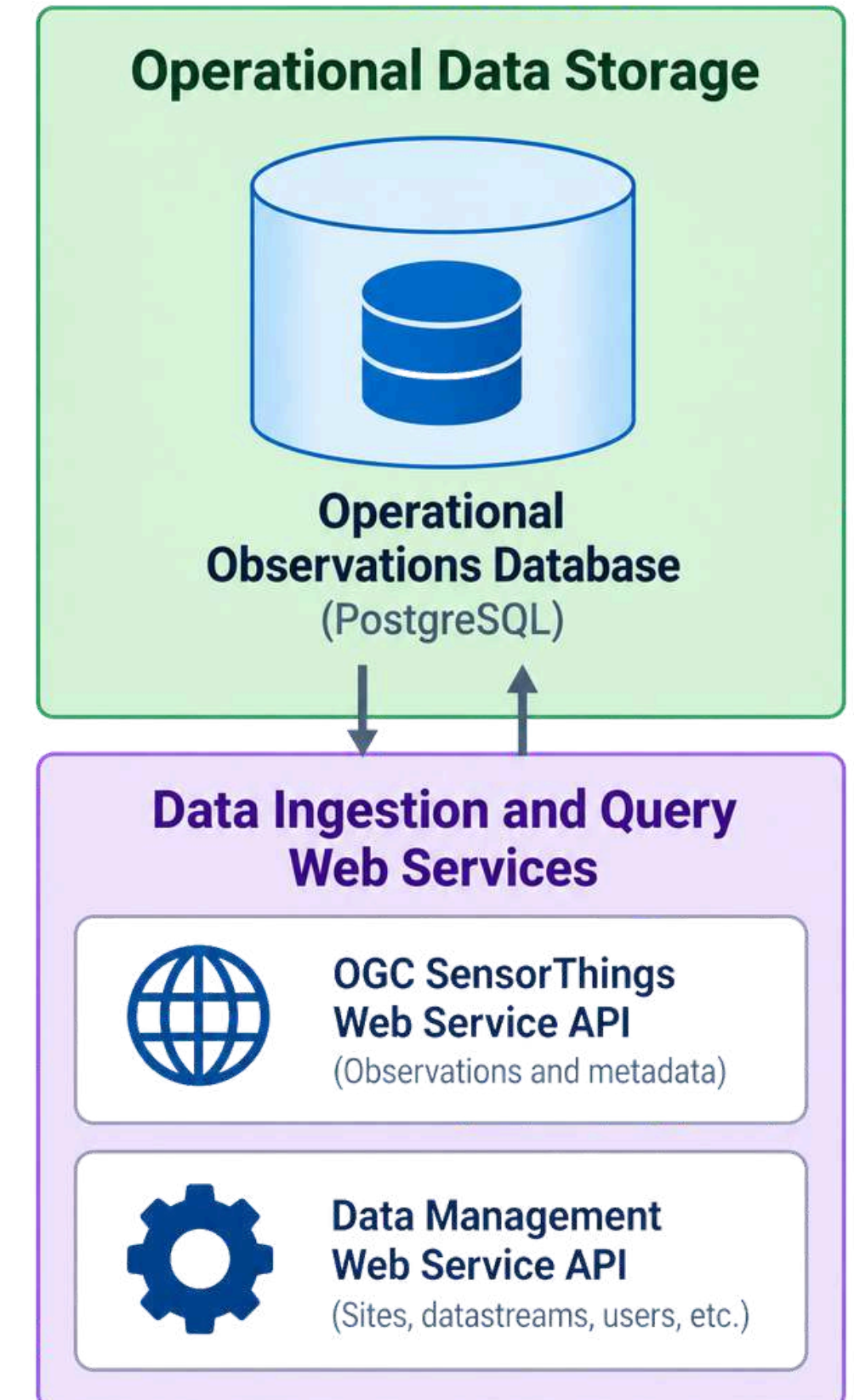
- Operational data storage provides the **location, structure, and technology for storing environmental observations and metadata.**



Today's Interactive Exercise!

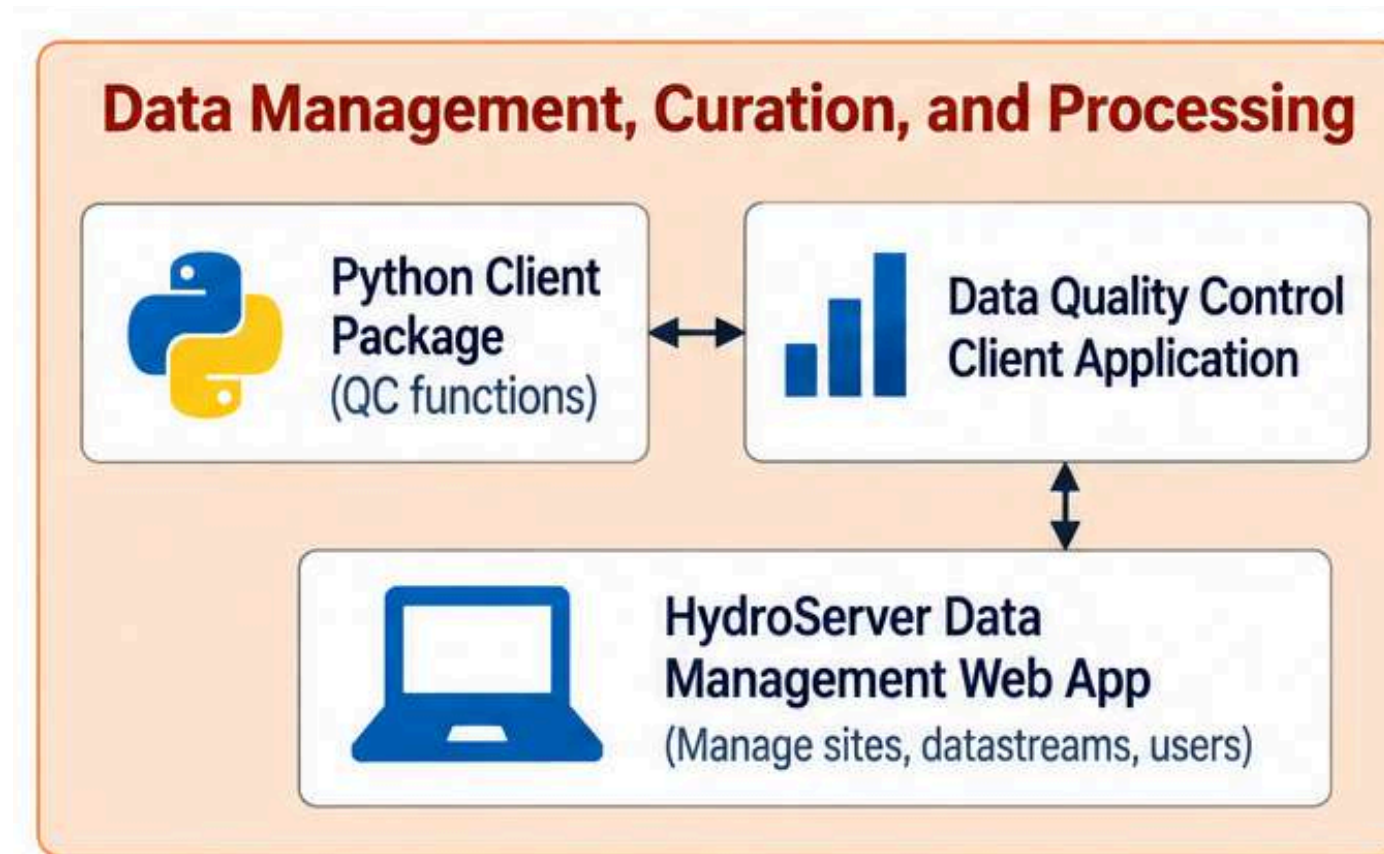


- Data model: OGC SensorThings (STA) + Observations Data Model (ODM).
- Uses this data model for both data storage and data exchange through APIs.
- Enables multi-user data management through users, workspaces, and access control.



Data Management, Curation, and Processing

- **Web-based interface** for managing users, metadata, access control, and data loading.
- All management functions can also be automated through the web service APIs.
- Provides **quality control (QC) tools** for validating and processing raw observations



Design of data publication, sharing, and exchange

- Publishes and shares data through the OGC SensorThings (STA) API
- Python library simplifies data retrieval and analysis; Other programming languages can be used too.
- Open architecture **supports development** (i.e., external dashboards, visualization tools, etc.)
- HydroShare integration enables long-term data preservation & CSV downloads

Data Publication, Sharing and Exchange



External Websites with Visualization and Download
(e.g., dashboards, portals)



Python Client Package (Query Functions)
(Access and retrieve data)



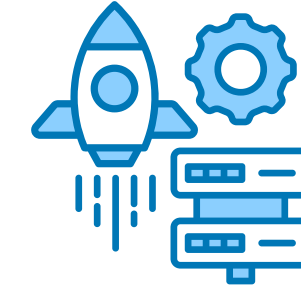
Catalog Services
(e.g., GeoNetwork, CKAN)



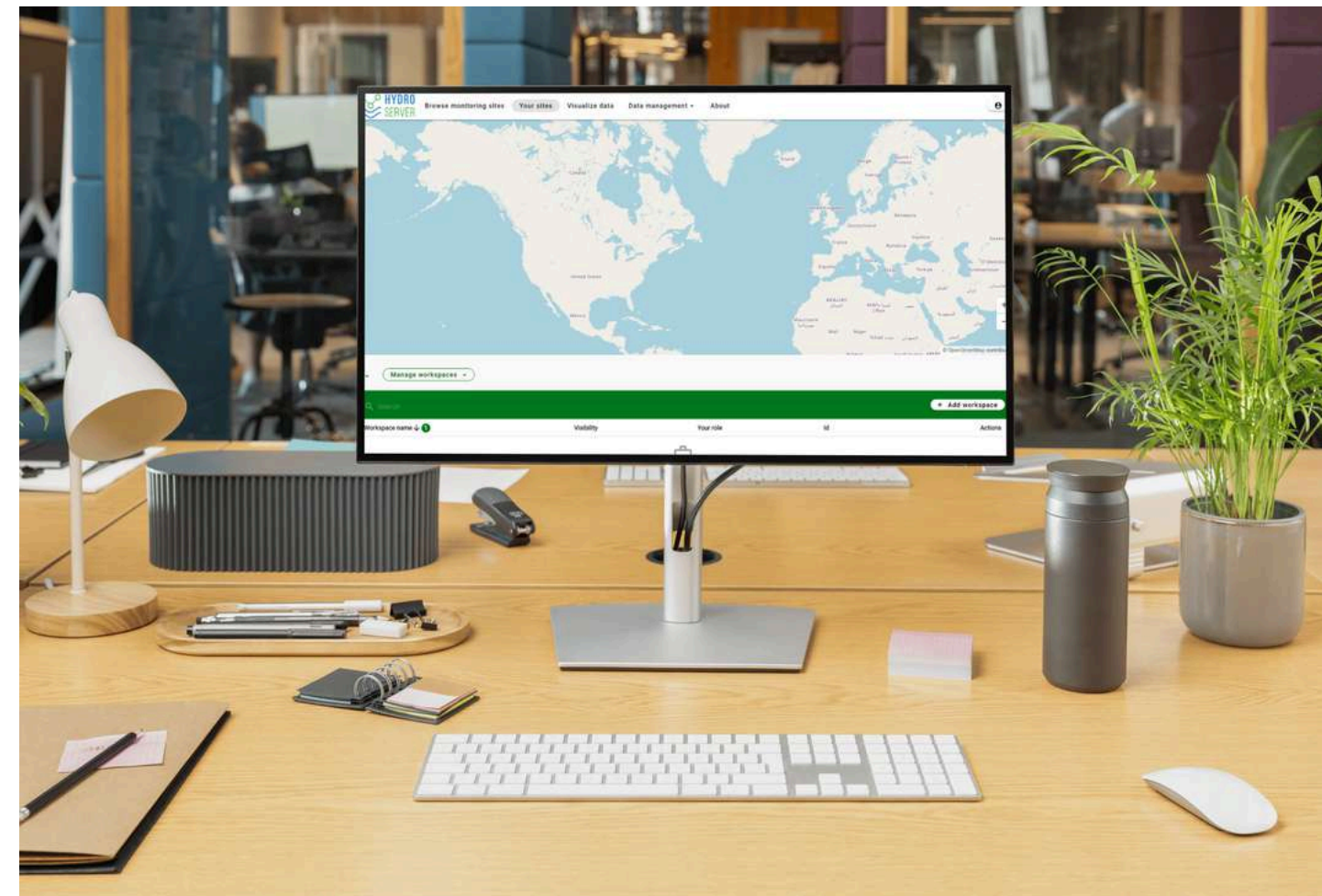
Archival in HydroShare Repository
(Long-term data preservation)

HydroServer Deployment

HydroServer Deployment



Deploying HydroServer means installing and configuring the HydroServer software stack on infrastructure managed by your organization, either in the cloud or on a local/on-premises server.



HydroServer Playground vs. Deployment

- The Playground is a shared HydroServer environment for demonstration and training.
- For operational use, organizations can deploy HydroServer on their own infrastructure.
- This provides a **dedicated database, APIs, web applications, users, and job orchestration for automated ETL workflows.**



HydroServer Deployment



- HydroServer is intentionally **decentralized**, so each organization can operate its own persistent instance.
- You manage it yourself either **on-premises** or in a **cloud environment such** as Google Cloud Platform.



On-cloud vs On-premise

- **On cloud deployment** means that HydroServer is hosted on servers provided by a cloud company
- **On-premise deployment** means hosting HydroServer on servers owned and managed by your organization



On-cloud vs On-premise



Runs on a **cloud provider**

Runs on your **own infrastructure**

Easier to **scale and access remotely**

Greater **control over data & infrastructure**

Requires reliable **internet connectivity**

Can operate within an **internal network**

Provider manages **physical infrastructure**

Organization manages **hardware & maintenance**

Generally **faster to deploy**

May require more **local IT expertise**

Ongoing cloud costs

Upfront hardware costs

Illustrative Google Cloud Costs

	Component	Example configuration	Approx. cost
	Compute	E2-standard-4 — 4 vCPU, 16 GB RAM	\$107.6245716/month
	Storage	500 GB Balanced Persistent Disk	~\$54.89/month
	Estimated monthly	Compute + storage	~\$162.51/month
	Estimated annual		~\$1,950/year

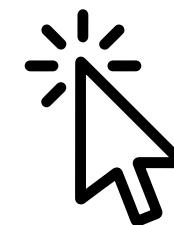
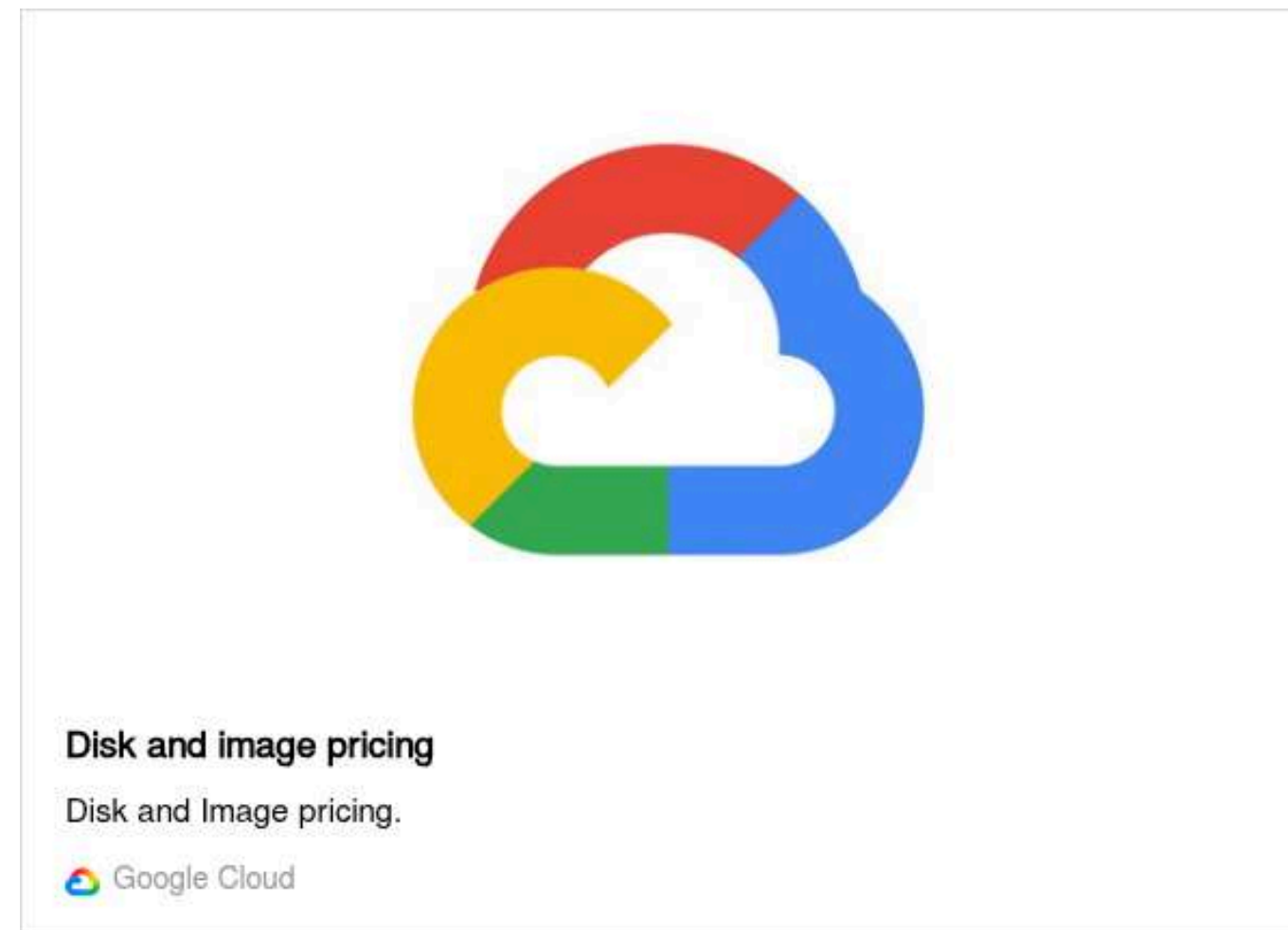


Note: These prices are estimates based on Google Cloud pricing information. Do not consider them the final cost of your HydroServer deployment. Actual costs will depend on your specific infrastructure, configuration, data volume, and usage

Explore Google Cloud Pricing



To learn more about deployment costs, visit the official Google Cloud pricing page.



Deployment with Google Cloud

The next slides present the step-by-step process I followed to deploy HydroServer on Google Cloud.



We will not perform the deployment during this training, but you can use Slides 48–64 as a guide to deploy your own HydroServer instance.

Step 1: Sign in with your Google account



Go to: <https://console.cloud.google.com/welcome/>

and sign in with your google account

Step 2: Create or choose your Google Cloud project

 Start your Free Trial with \$300 in credit. Don't worry—you won't be charged if you run out of credits. [Learn more](#)

Dismiss [Start free](#)

Google Cloud

Select a project

Search (/) for resources, docs, products, and more

Search

Dismiss

Start free

Open project picker (Ctrl O)

Then Click in Select a project

Web

Vicario

Try Google Cloud with \$300 in free credits

✓ Access to Google Cloud products and services

✓ 90 days to spend your credits

✓ No billing during trial

Try for free

Other options

Try the latest Gemini models

[Try Gemini](#)

Configure Google Cloud for scalable, production-ready enterprise workloads.

[Google Cloud setup](#)

+ Create Gemini API key

+ Create an agent

+ Create a VM

+ Run a query in BigQuery

+ Deploy an application

+ Create a storage bucket

Popular getting started resources

Filter by

Data, Agents, and AI/ML

Web, Mobile, Game, Storage

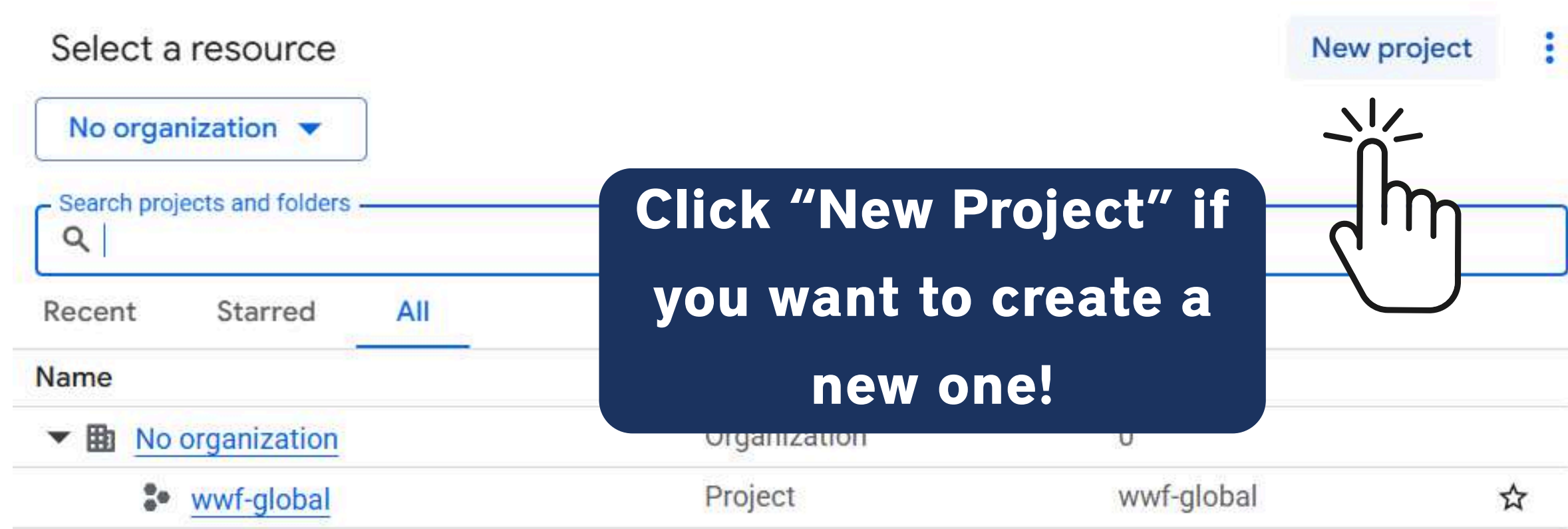
Containers, VMs, Hybrid/Multi, Move Workload

Maps, APIs

General

Products

Step 2: Create or choose your Google Cloud project



Step 2: Fill out information about your Google Cloud project

Give a name to your project

Select a billing account. **A billing account is required to deploy the Google Cloud resources used by HydroServer.**

Note: Google Cloud currently offers \$300 in free credits for new customers!

New Project

Fill out the information

Project name *

hydroserver-test-sara

Project ID: hydroserver-test-sara. It cannot be changed later. Edit

Billing account *

My Billing Account

Any charges for this project will be billed to the account you select here.

Organization *

svicario-org

Select an organization to attach it to a project. This selection can't be changed later.

Parent resource *

 svicario-org Browse

Select the organization or folder where this resource will be created

Create

Cancel



When you are finished, click Create.

Your project has been created successfully!

redits, then pay only for what you use.

Dismiss

Activate

Notifications

- ✓ Create Project: hydroserver-test-sara Just now
[Select Project](#)
- ✓ Create Organization and Project 46 minutes ago

[See all activities](#)



Step 3: Activate the Cloud Shell

Google Cloud

svicario-org

Search (/) for resources, docs, products, and more

Search

Activate Cloud Shell (G then S)

Welcome, Sara Vicario

Create or select a project to start using Google Cloud

[Learn more about projects](#)

Create or select a project

Try Gemini Cloud Assist chat

Chat now

+ Create Gemini API key

+ Create an agent

+ Create a VM

+ Run a query in BigQuery

+ Deploy an application

+ Create a storage bucket

See recommendations based on your interests in [Data, Agents, and AI/ML](#)

Products

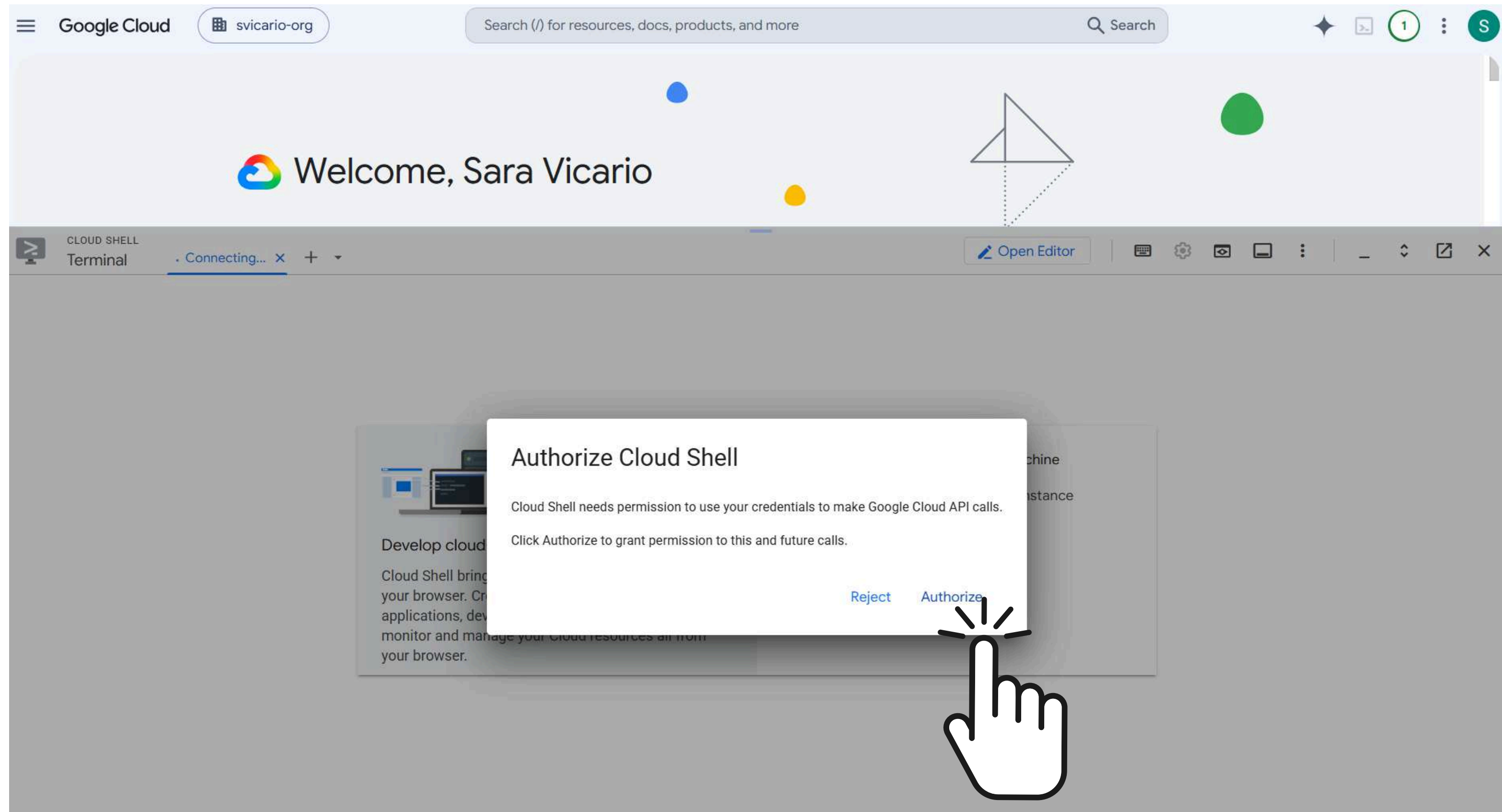
Build an agent

Try the latest Gemini

API Get Agent Platform API

Explore 200+

Step 3: Activate the Cloud Shell



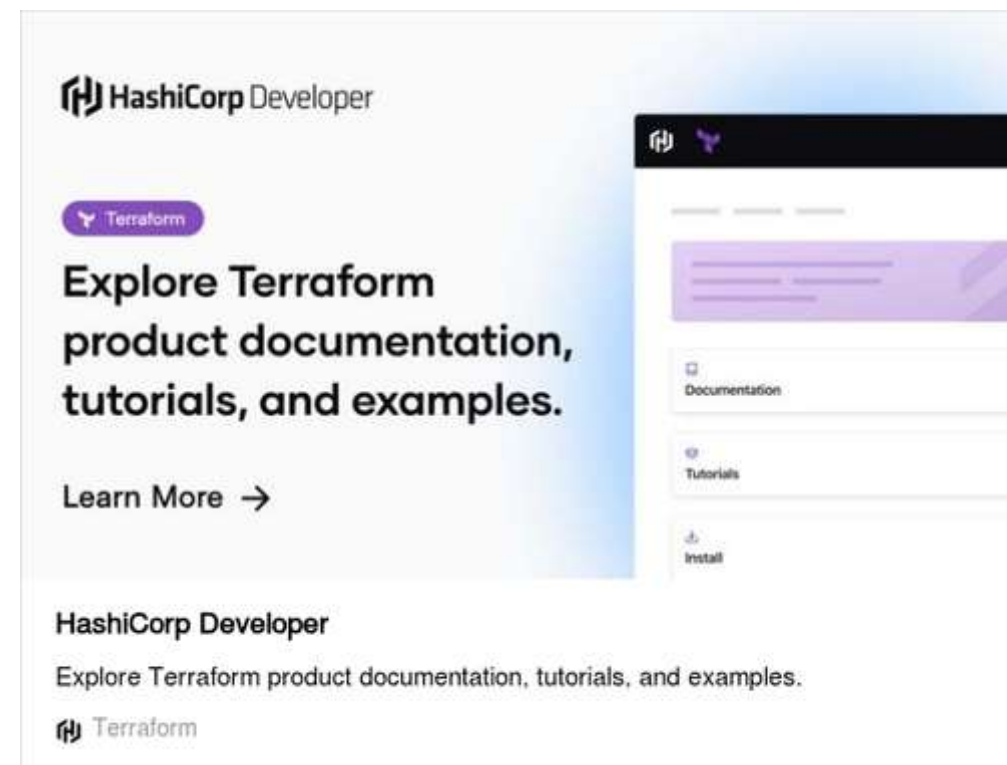
Step 3: Set the Active Google Cloud Project

Set your HydroServer project

```
$ gcloud config set project hydroserver-test-sara
```

Step 4: Install Terraform

Terraform is an Infrastructure as Code tool. It automatically creates the Google Cloud resources needed to deploy HydroServer.



Step 4: Install Terraform

- Add HashiCorp's security key – verifies that the Terraform package comes from a trusted source.

```
wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
```

- Add the HashiCorp repository – tells Ubuntu where to find Terraform.

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]  
https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" |  
sudo tee /etc/apt/sources.list.d/hashicorp.list
```

- Update available packages

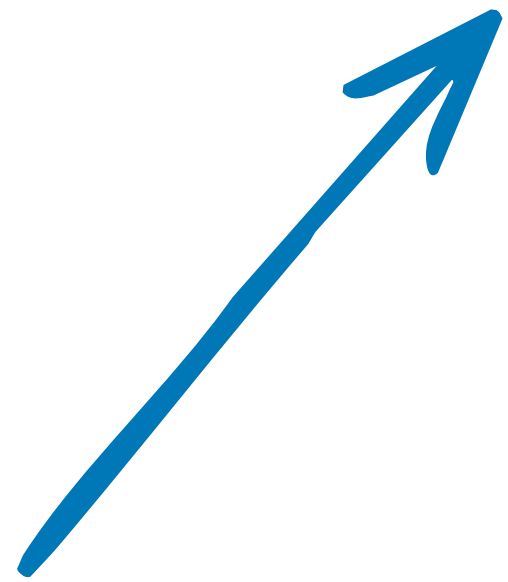
```
sudo apt update
```

- Install Terraform

```
sudo apt install terraform
```

Step 4: Verify the Terraform installation

```
svicario@cloudshell:~ (hydroserver-test-sara)$ terraform --version  
Terraform v1.16.0  
on linux_amd64
```



This shows the Google console

Step 5: Created a folder for the HydroServer deployment

```
mkdir hydroserver-gcp  
cd hydroserver-gcp
```

```
svicario@cloudshell:~ (hydroserver-test-sara)$ mkdir hydroserver-gcp  
cd hydroserver-gcp  
svicario@cloudshell:~/hydroserver-gcp (hydroserver-test-sara)$
```

The project folder has been created and set as directory

Step 6: Initiate Terraform

\$ terraform int

```
svicario@cloudshell:~/hydroserver-gcp (hydroserver-test-sara)$ terraform init
Initializing the backend...
```

Initializing provider plugins...

- Finding latest version of hashicorp/null...
- Finding latest version of hashicorp/random...
- Finding hashicorp/google versions matching "~> 6.0"...
- Installing hashicorp/null v3.3.1...
- Installed hashicorp/null v3.3.1 (signed by HashiCorp)
- Installing hashicorp/random v3.9.0...
- Installed hashicorp/random v3.9.0 (signed by HashiCorp)
- Installing hashicorp/google v6.50.0...
- Installed hashicorp/google v6.50.0 (signed by HashiCorp)

Terraform has created a lock file `.terraform.lock.hcl` to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
svicario@cloudshell:~/hydroserver-gcp (hydroserver-test-sara)$
```

Step 6: Enable Google Cloud APIs

Enable the Google Cloud APIs required by Terraform to create and manage the resources needed for the HydroServer deployment, including Cloud Run, Cloud SQL, Artifact Registry, IAM, and Secret Manager.

```
gcloud services enable \
cloudresourcemanager.googleapis.com \
artifactregistry.googleapis.com \
sqladmin.googleapis.com \
iam.googleapis.com \
secretmanager.googleapis.com \
run.googleapis.com \
--project=hydroserver-test-sara
```

Step 7: Run the Terraform Deployment

Run the HydroServer Terraform deployment

```
terraform apply \  
-var "region=us-west1" \  
-var "project_id=hydroserver-test-sara"
```

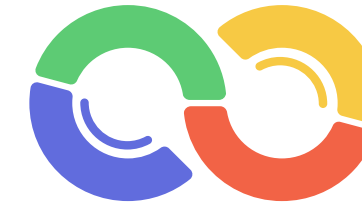
```
Do you want to perform these actions?  
Terraform will perform the actions described above.  
Only 'yes' will be accepted to approve.  
  
Enter a value: █
```

Enter yes





HydroServer is deployed



```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

google_cloud_run_v2_service.hydroserver: Destroying... [id=projects/hydroserver-test-sara/locations/us-west1/services/hydroserver-demo]
google_cloud_run_v2_service.hydroserver: Still destroying... [id=projects/hydroserver-test-sara/locations/us-west1/services/hydroserver-demo, 00m10s elapsed]
google_cloud_run_v2_service.hydroserver: Destruction complete after 12s
google_cloud_run_v2_service.hydroserver: Creating...
google_cloud_run_v2_service.hydroserver: Still creating... [00m10s elapsed]
google_cloud_run_v2_service.hydroserver: Still creating... [00m20s elapsed]
google_cloud_run_v2_service.hydroserver: Still creating... [00m30s elapsed]
google_cloud_run_v2_service.hydroserver: Still creating... [00m40s elapsed]
google_cloud_run_v2_service.hydroserver: Creation complete after 44s [id=projects/hydroserver-test-sara/locations/us-west1/services/hydroserver-demo]
google_cloud_run_service_iam_member.hydroserver_cloud_run_public_access: Creating...
google_cloud_run_service_iam_member.hydroserver_cloud_run_public_access: Creation complete after 7s [id=v1/projects/hydroserver-test-sara/locations/us-west1/services/hydroserver-demo/invoker/allUsers]

Apply complete! Resources: 2 added, 0 changed, 1 destroyed.

Outputs:

admin_password = <sensitive>
admin_username = "admin@hydroserver.org"
database_url = <sensitive>
service_url = "https://hydroserver-demo-998103630590.us-west1.run.app"
svicario@cloudshell:~/hydroserver-gcp (hydroserver-test-sara)$
```



 Congratulations! You now have your own HydroServer instance running on Google Cloud.

hydroserver-demo-998103630590.us-west1.run.app/browse

 **HYDRO SERVER**

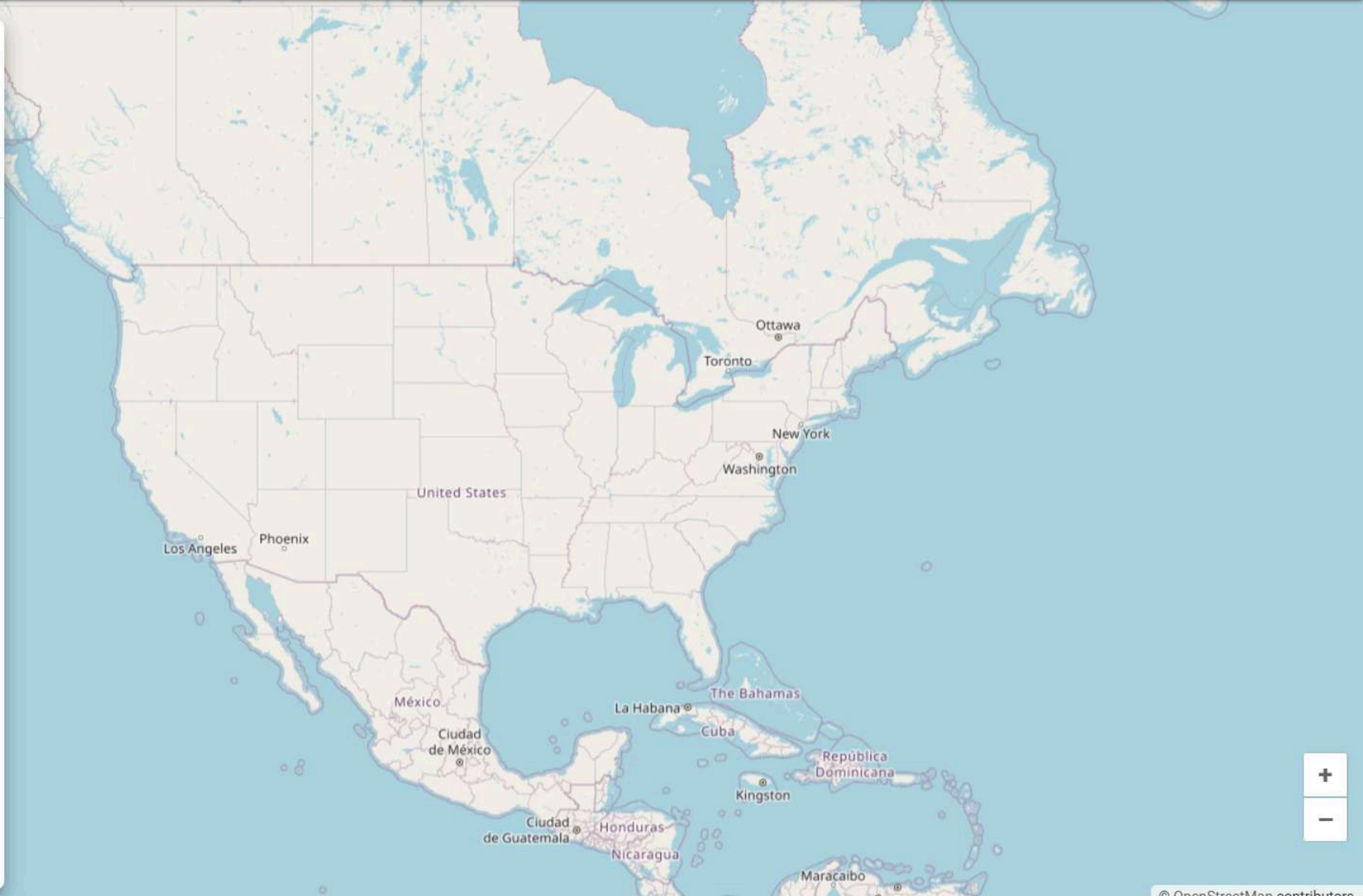
[Browse monitoring sites](#) [Your sites](#) [Visualize data](#) [Data management](#) [About](#) [Log in](#)

Monitoring sites

Workspaces

0 SITES

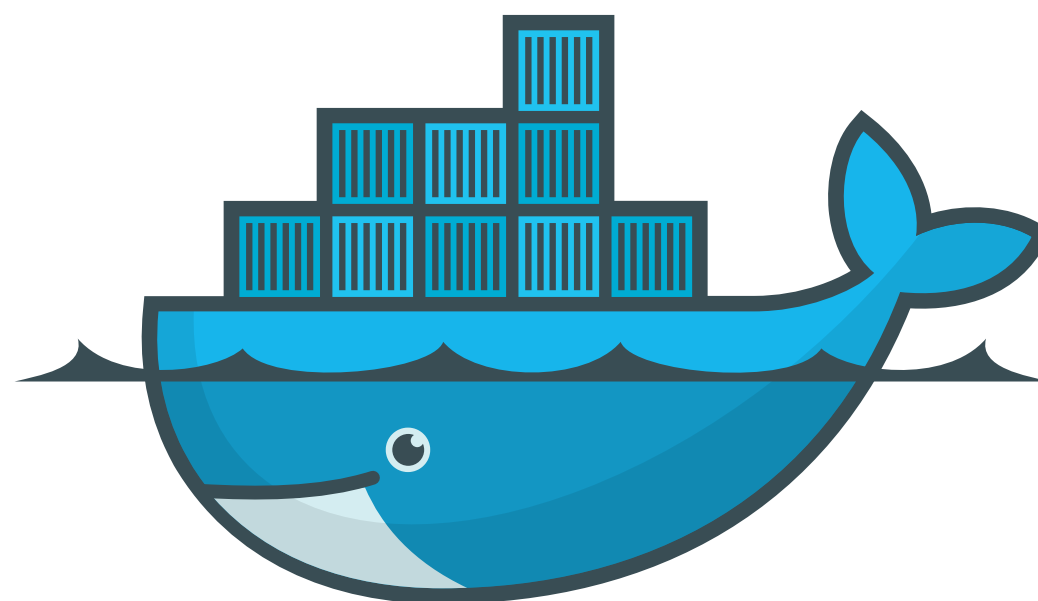
No sites match these filters.



© OpenStreetMap contributors.

Deployment with Docker

The next slides present the step-by-step process
I followed to deploy HydroServer on Docker



We will not perform the deployment during this training, but you can use Slides 67–79 as a guide to deploy your own HydroServer instance.

Step 1: Check the Installed Docker Version

On Windows, open Windows PowerShell and check your Docker version:

`docker --version`

```
PS C:\Windows\system32> docker --version  
Docker version 29.2.1, build a5c7197  
PS C:\Windows\system32> |
```



Step 2: Create a folder in your computer

Create a folder, for instance in the Disk C:

C:\hydroserver



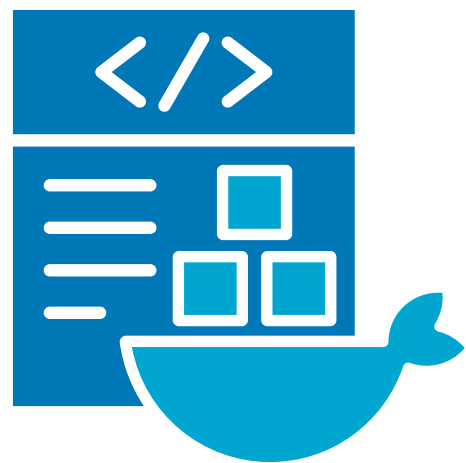
Then, navigate to this folder in PowerShell:

```
PS C:\Windows\system32> cd C:\hydroserver|
```

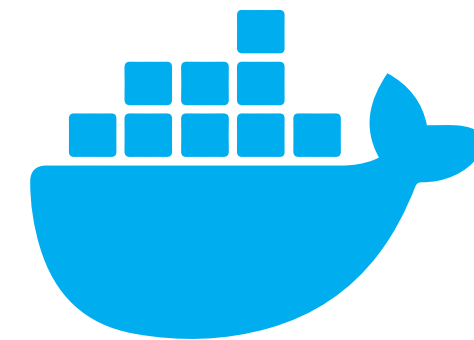


Step 3: Download the Docker Compose File from GitHub

Download the official docker-compose.yaml file from the HydroServer GitHub repository:

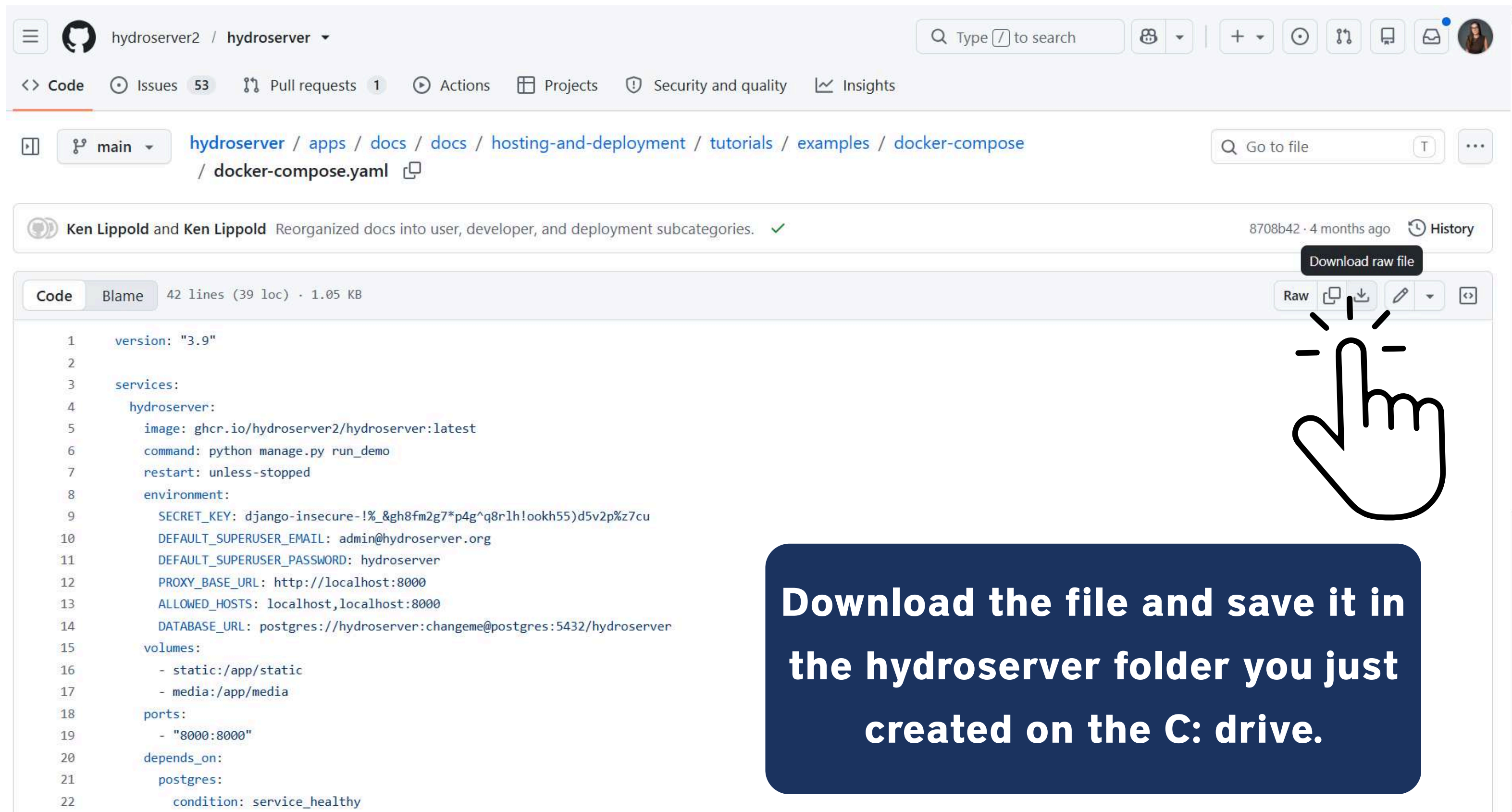


docker-compose.yaml



See the next slide to see where to click to download the file.

Step 3: Download the Docker Compose File from GitHub



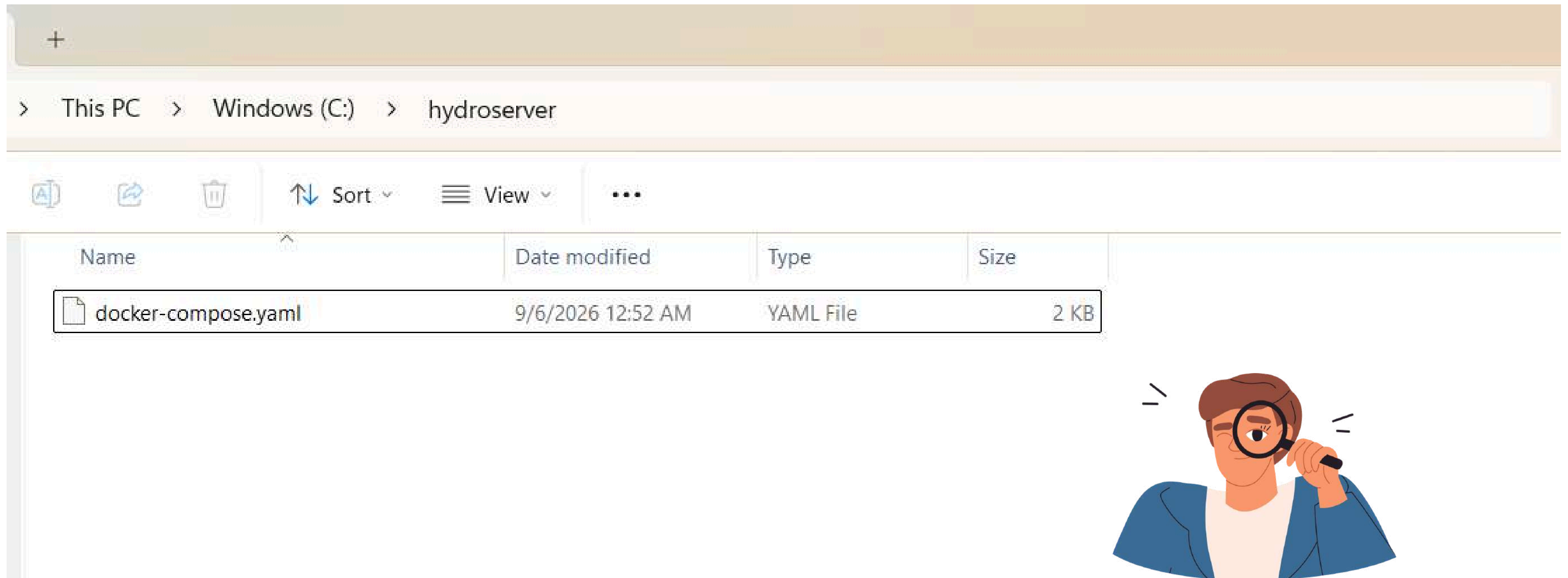
The screenshot shows the GitHub interface for the repository `hydroserver2 / hydroserver`. The breadcrumb path is `hydroserver / apps / docs / docs / hosting-and-deployment / tutorials / examples / docker-compose / docker-compose.yml`. A commit by Ken Lippold is shown. The file `docker-compose.yml` is displayed with 42 lines. A hand icon points to the `Download raw file` button in the top right of the file view area.

```
1  version: "3.9"
2
3  services:
4    hydroserver:
5      image: ghcr.io/hydroserver2/hydroserver:latest
6      command: python manage.py run_demo
7      restart: unless-stopped
8      environment:
9        SECRET_KEY: django-insecure-!%_&gh8fm2g7*p4g^q8r1h!ookh55)d5v2p%z7cu
10       DEFAULT_SUPERUSER_EMAIL: admin@hydroserver.org
11       DEFAULT_SUPERUSER_PASSWORD: hydroserver
12       PROXY_BASE_URL: http://localhost:8000
13       ALLOWED_HOSTS: localhost,localhost:8000
14       DATABASE_URL: postgres://hydroserver:changeme@postgres:5432/hydroserver
15     volumes:
16       - static:/app/static
17       - media:/app/media
18     ports:
19       - "8000:8000"
20     depends_on:
21       postgres:
22         condition: service_healthy
```

Download the file and save it in the hydroserver folder you just created on the C: drive.

Step 3: Download the Docker Compose File from GitHub

It should look something like this!



Step 3: Download the Docker Compose File from GitHub

By typing `dir` in PowerShell, you can check that the file is there.



```
PS C:\Windows\system32> cd C:\hydroserver  
PS C:\hydroserver> dir
```

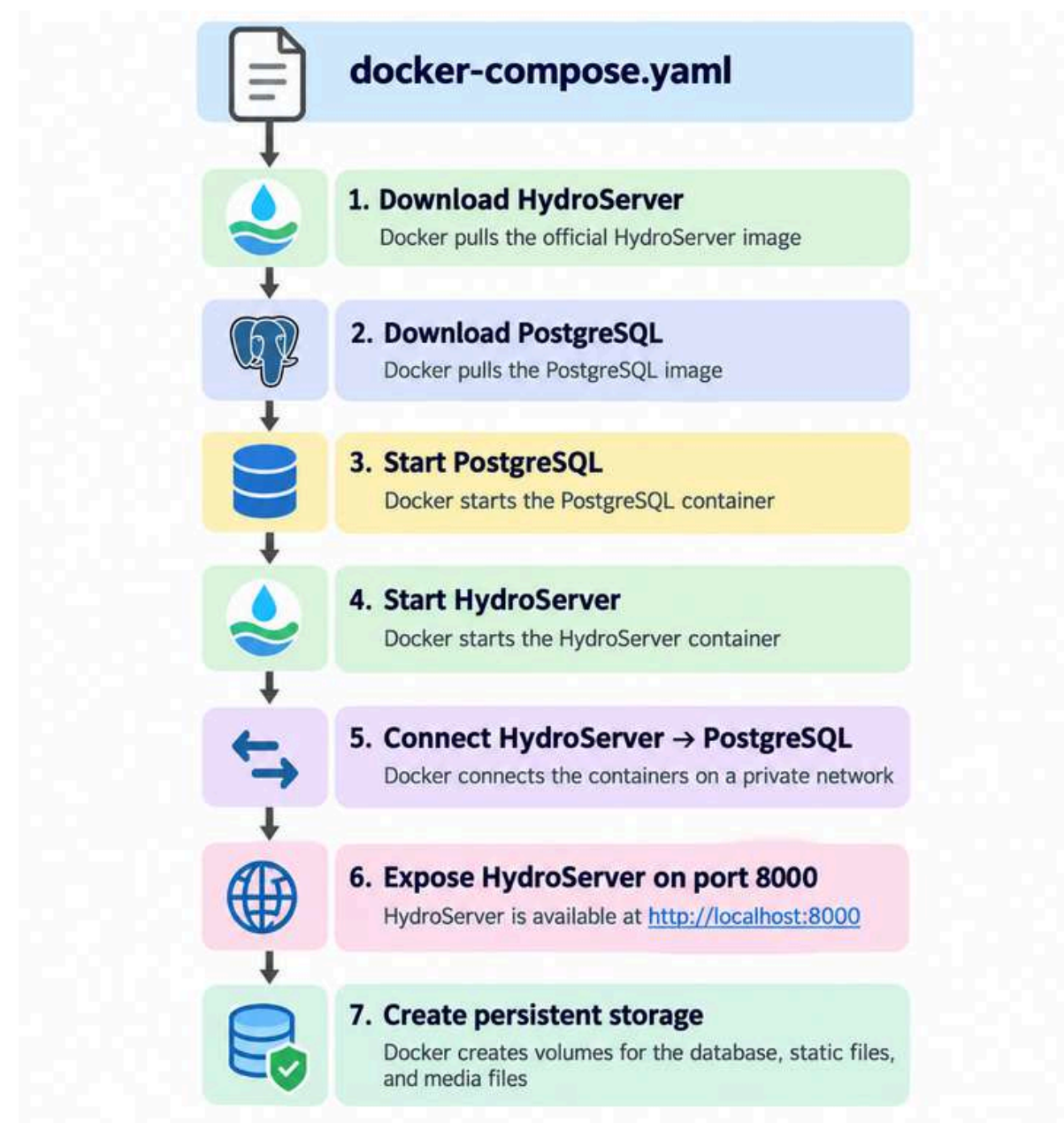
Directory: C:\hydroserver

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	9/6/2026 12:52 AM	1071	docker-compose.yaml



Step 3: Download the Docker Compose File from GitHub

Think of docker-compose.yaml as the instructions for Docker.
Inside it, HydroServer tells Docker roughly:



Step 4: Start HydroServer with Docker Compose

Now, run:

docker compose up

this reads the Docker Compose file and starts all the services needed to run HydroServer.

```
PS C:\hydroserver> docker compose up
time="2026-09-06T01:21:34-07:00" level=warning msg="C:\\hydroserver\\docker-compose.yaml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 20/31
- Image postgres:17 [.] 45.1MB / 167MB Pulling 5.0s
- Image ghcr.io/hydroserver2/hydroserver:latest [.] 32.51MB / 433.3MB Pulling 5.0s
```

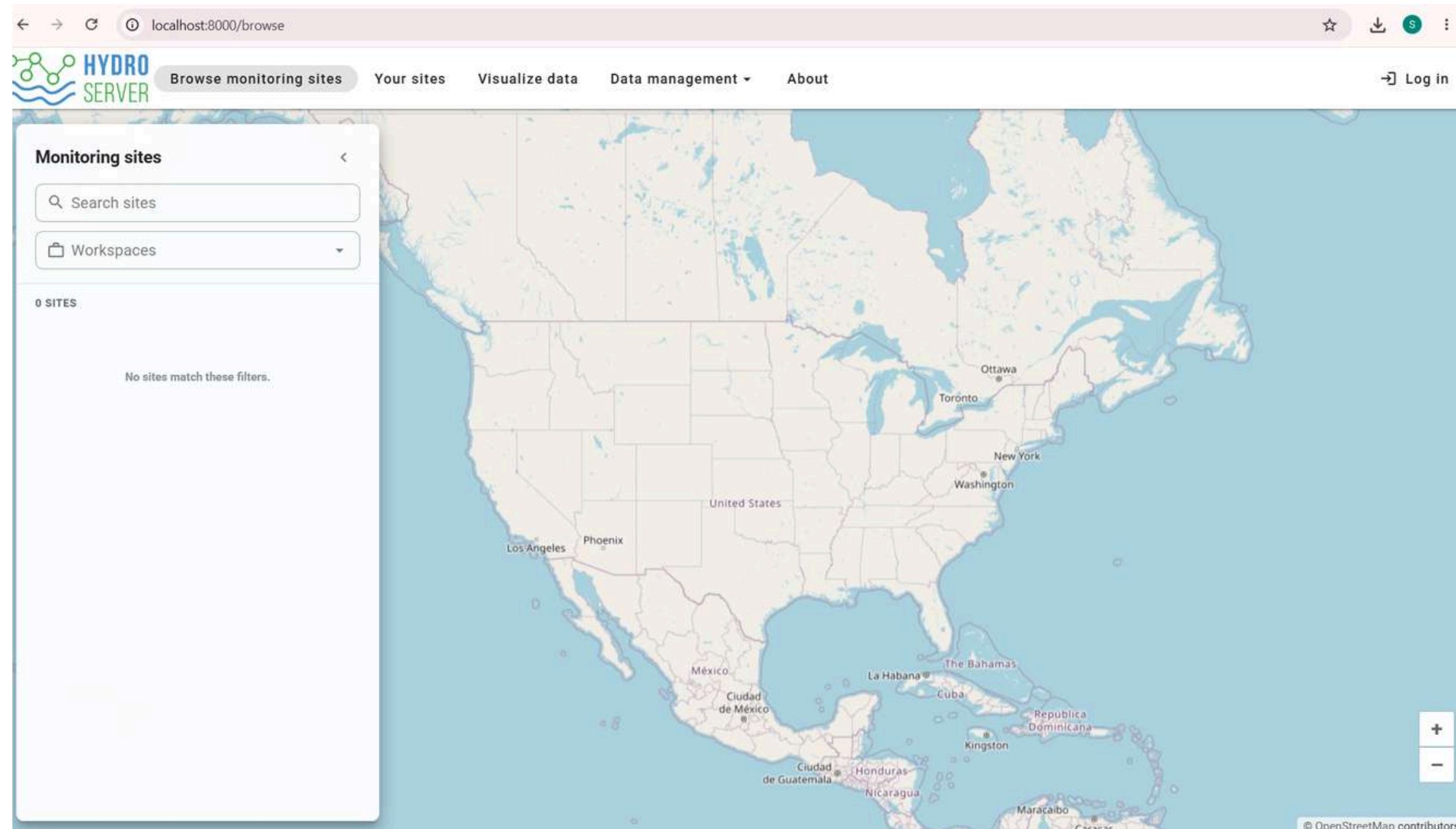
The first run may take a few minutes while Docker downloads the required images.

Step 5: Access Your HydroServer Deployment

Then, go to:



`http://localhost:8000`



Step 5: Access Your HydroServer Deployment

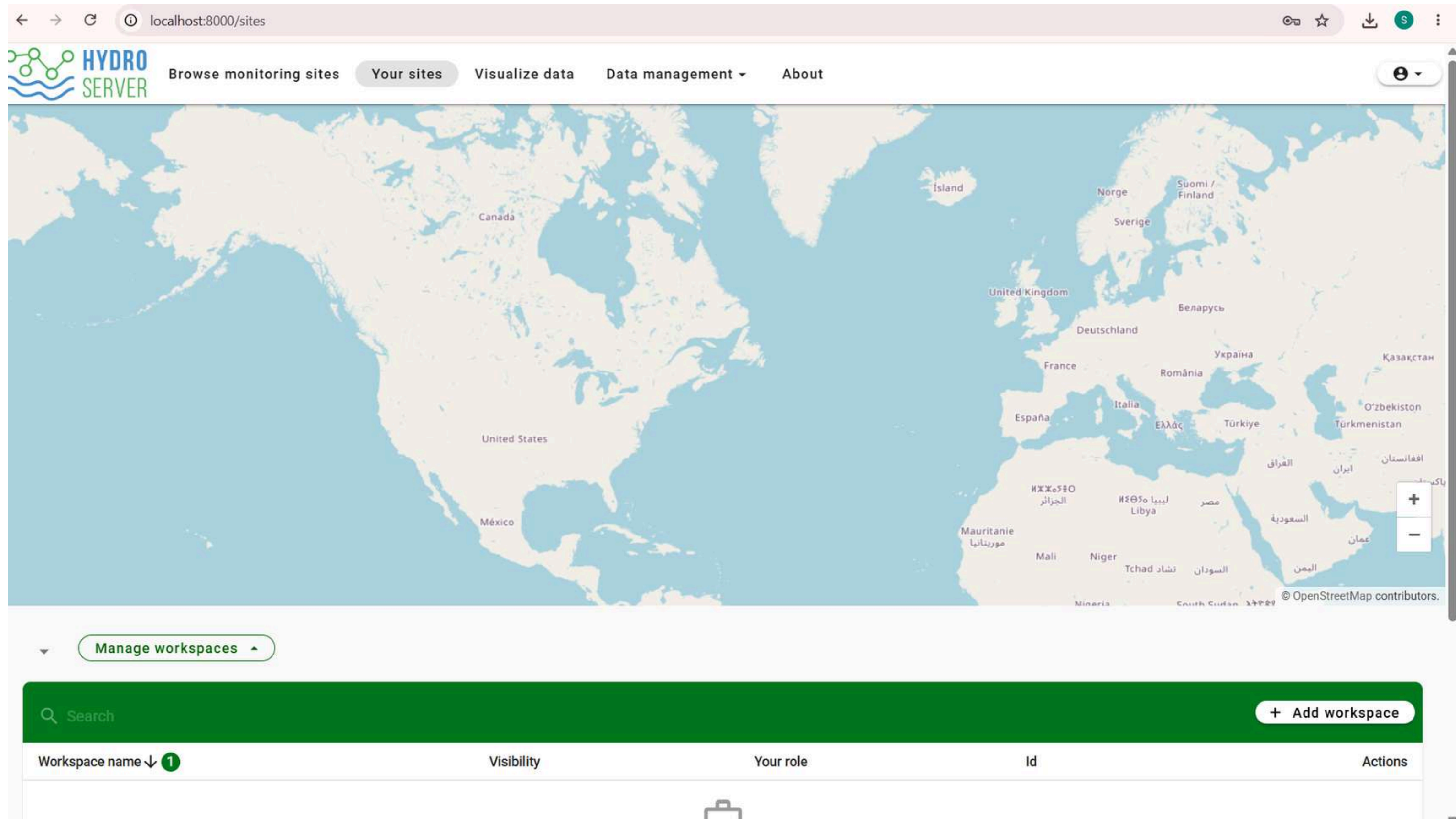
You can find the
credentials in the
`docker-compose.yaml`
file!

Log in with these credentials

`DEFAULT_SUPERUSER_EMAIL: admin@hydroserver.org`
`DEFAULT_SUPERUSER_PASSWORD: hydroserver`

A screenshot of the HydroServer web application's login page. The page has a light gray background. At the top left is the 'HYDRO SERVER' logo. To its right is a navigation bar with links: 'Browse monitoring sites', 'Your sites', 'Visualize data', 'Data management', and 'About'. On the far right of the navigation bar is a 'Log in' button. In the center of the page is a 'Log in' form. The form has two input fields: 'Email *' with the value 'admin@hydroserver.org' and 'Password *' with masked characters. Below the password field is a blue 'Log in' button. Under the form, there are two links: 'Don't have an account? Sign Up' and 'Forgot your password?'. At the bottom of the page, there is a footer. On the left, it says 'Developed by' followed by the Utah State University logo and 'Utah Water Research Laboratory UtahStateUniversity'. On the right, it says 'Powered by' followed by the 'HYDRO SERVER' logo.

You have successfully logged in! 🎉



🎉 Your HydroServer instance is now deployed and ready to use! 🎉

When you're finished testing, you can stop the deployment. In the PowerShell window where docker compose up is running, press:

Ctrl + C

```
Gracefully Stopping... press Ctrl+C again to force
Container hydroserver-hydroserver-1 Stopping
Container hydroserver-hydroserver-1 Stopped Enable Watch [d] Detach
Container hydroserver-postgres-1 Stopping
Container hydroserver-postgres-1 Stopped
```

Then run:

docker compose down

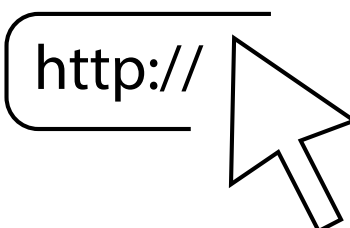
```
PS C:\hydroserver> docker compose down [w] Enable Watch [d] Detach
time="2026-09-06T01:51:03-07:00" level=warning msg="C:\\hydroserver\\docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] down 3/3
✓Container hydroserver-hydroserver-1 Removed 0.1s
✓Container hydroserver-postgres-1 Removed 0.0s
✓Network hydroserver_default Removed 0.2s
```


Your data should remain in the persistent Docker volumes. The next time you want HydroServer, from C:\hydroserver simply run:

docker compose up

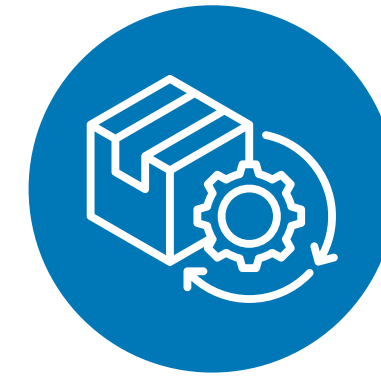
```
PS C:\hydroserver> docker compose up
time="2026-09-06T01:52:12-07:00" level=warning msg="C:\\hydroserver\\docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 3/3
  ✓Network hydroserver_default          Created      0.0s
  ✓Container hydroserver-postgres-1     Created      0.1s
  ✓Container hydroserver-hydroserver-1 Created      0.1s
Attaching to hydroserver-1, postgres-1
Container hydroserver-postgres-1 Waiting
postgres-1 |
postgres-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
postgres-1 |
postgres-1 | 2026-09-06 08:52:13.418 UTC [1] LOG:  starting PostgreSQL 17.11 (Debian 17.11-1.pgdg13+2) on x86_64-pc-lin
ux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
postgres-1 | 2026-09-06 08:52:13.420 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
postgres-1 | 2026-09-06 08:52:13.420 UTC [1] LOG:  listening on IPv6 address ":::", port 5432
postgres-1 | 2026-09-06 08:52:13.424 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
postgres-1 | 2026-09-06 08:52:13.433 UTC [29] LOG:  database system was shut down at 2026-09-06 08:50:13 UTC
postgres-1 | 2026-09-06 08:52:13.442 UTC [1] LOG:  database system is ready to accept connections
```

and access **http://localhost:8000** again.



Production Configuration

| Production Configuration



Once HydroServer is deployed ,**production configuration** means configuring that deployment so it is secure, reliable, persistent, and suitable for real users and operational data. It is not a second HydroServer installation.

The next slide summarizes the main steps for setting up HydroServer in a **production environment**. We will not cover these steps during this training, but you can use them as a guide if you want to deploy your own HydroServer instance.

HydroServer provides detailed, step-by-step documentation for production deployment, which you can access [here](#).

[Setting Up a Production Deployment](#)



Setting Up a Production Deployment

- **Configure production environment variables** – SECRET_KEY, DEBUG=False, DATABASE_URL, PROXY_BASE_URL, account settings, email settings, storage settings, etc.
- **Configure PostgreSQL 15+** – provide sufficient CPU, memory, disk performance, backups, and monitoring.
- **Configure static & media file storage** – e.g., Google Cloud Storage for a GCP deployment.
- **Configure email** – connect an external SMTP service for verification and password resets.
- **Enable HTTPS** – production HydroServer should be served exclusively over HTTPS.
- **Configure load balancing & scalability, if needed** – for high numbers of concurrent requests.
- **Configure database connection pooling, if needed** – HydroServer suggests tools such as PgBouncer for larger deployments.
- **Configure security & access control** – restrict database/network access, use least privilege, secure communications, and keep software updated.
- **Configure monitoring & logging** – including HydroServer's /health-check endpoint.
- **Enable Celery, if needed** – worker + scheduler + broker such as Redis for background/scheduled tasks.

Deployment Considerations

- **Deployment is more than running Docker containers:** production environments also require decisions about networking, availability, and scalability.
- **Scale according to expected usage:** consider the number of users, API requests, and overall workload.
- Different HydroServer components, such as the **frontend and API/backend, may have different scaling needs.**
- For larger deployments, technologies such as load balancers and Kubernetes can help distribute traffic and manage containers.
- **Containers provide flexibility:** HydroServer can be deployed using different cloud services and architectures depending on your requirements and budget.

If you have specific questions about deployment, Jorge, who will join us online during the last session, will be able to help you more



HydroServer Data Model



What Is a Data Model?



- A **data model** defines how data are organized, structured, and related.
- It provides a **common vocabulary** for describing data.
- For environmental monitoring, it **defines relationships between monitoring sites, sensors, variables, and observations.**
- Using a **standardized data model** makes data easier to manage, query, share, and integrate.





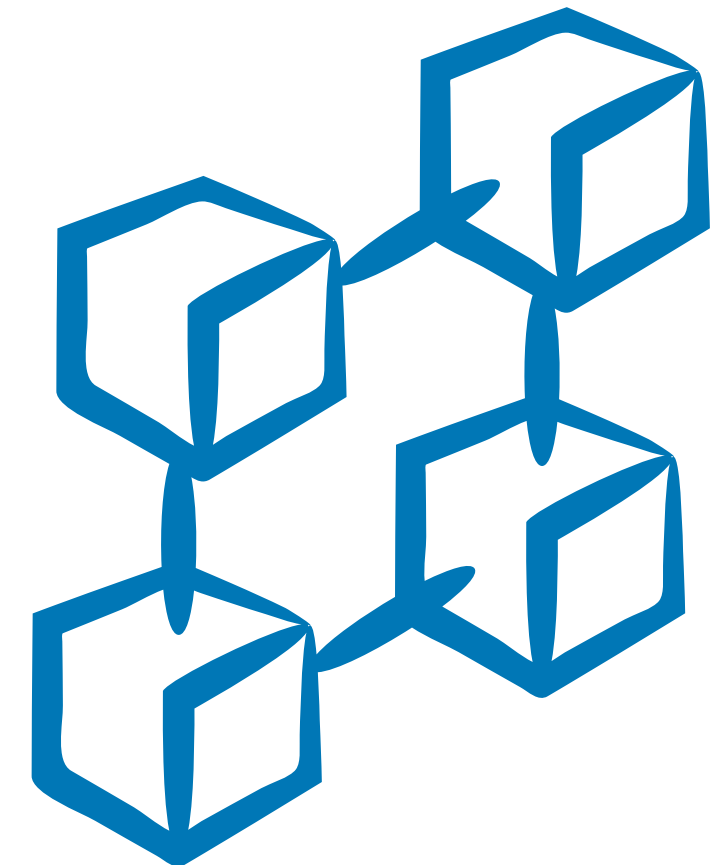
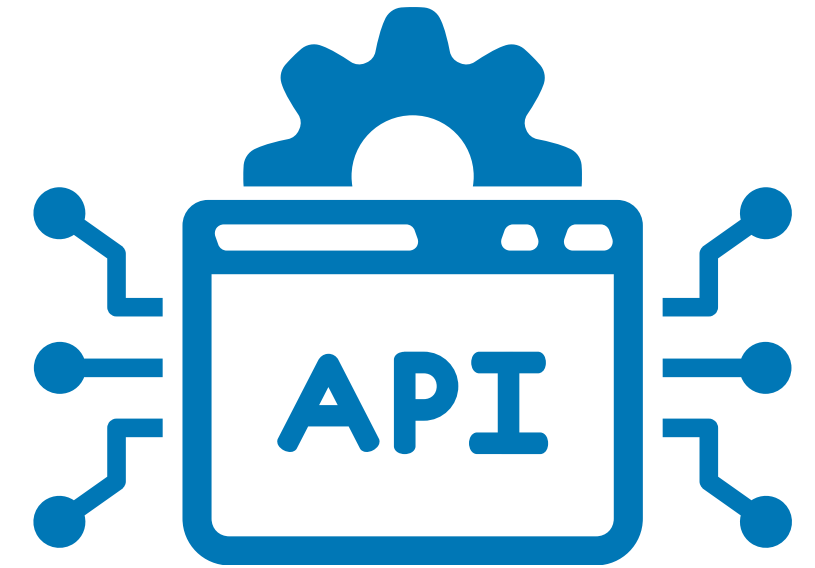
HydroServer is built on open standards



- HydroServer uses the **Open Geospatial Consortium SensorThings Application Programming Interface (OGC SensorThings API)** as the foundation for its data model and standardized access to sensor observations.
- HydroServer extends the SensorThings model with additional metadata and functionality, drawing on concepts from the **Observations Data Model (ODM)** to better support hydrologic and environmental monitoring data.

OGC SensorThings API

- OGC SensorThings API is an **open, international standard** for managing and sharing sensor observations.
 - It provides a **standardized data model** for organizing monitoring sites, sensors, measured variables, and observations.
 - It provides an **API** for accessing and sharing these data in an interoperable way.



Definition

- **OGC SensorThings (STA) data model:** An OGC standard that defines how sensor observations, metadata, and their relationships are organized and exchanged
- **Observations Data Model (ODM):** A hydrology-specific data model that adds metadata such as units, methods, and quality control.
- **Application Programming Interface (API):** set of rules that allows one application to request or send data to another application.

Example

SensorThings STA:

- Thing: River gauge station
- Location: Nzoia River, Kenya
- Sensor: Water level sensor
- Observed Property: River stage
- Observation: 2.35 m at 10:00 AM

Additional metadata from ODM:

- Unit: m
- Processing Level: Quality controlled
- Result Qualifier: Verified
- Method: Pressure transducer

Retrieve observations from Datastream 1.

- [https://www.hydroserver.com/api/sensorthings/v1.1/Datastreams\(1\)/Observations](https://www.hydroserver.com/api/sensorthings/v1.1/Datastreams(1)/Observations)

Mapping Monitoring Site Metadata to the HydroServer Data Model

Monitoring site metadata you may already have	HydroServer data model property	Example for Kanzenze	What it means
Station name	name	Kanzenze Hydrological Station	Human-readable station name
Station description	description	Hydrological monitoring station on the Nyabarongo River	Description of the site
Feature type	sampling_feature_type	Site	Type of sampling feature
Station ID / Station code	sampling_feature_code	259501	Organization's identifier for the station
Site type	site_type	Stream	Type of monitoring site
Latitude	latitude	-2.0613	Latitude
Longitude	longitude	30.0877	Longitude
Elevation	elevation_m	1338	Elevation in meters
Elevation reference	elevation_datum	EGM96	Datum used for elevation
Province / State / Region	admin_area_1	Eastern Province	First administrative area
District / County	admin_area_2	Bugesera	Second administrative area
Country	country	RW	ISO 2-letter country code
Data-use warning	data_disclaimer	Data may be provisional	Disclaimer associated with the site
Public/private	is_private	FALSE	Controls whether the site is publicly accessible
Other organization-specific metadata	tags	network: Rwanda Hydromet	Custom key:value metadata

Mapping your Time-Series Metadata to the HydroServer Data Model

Time series metadata you may already have	HydroServer data model property	Example for Kanzenze	What it means
Time-series name	name	Water Level – Telemetry	Name of the datastream
Description	description	Water level observations at Kanzenze	Description of the time series
Monitoring station	thing	Kanzenze Hydrological Station	Site where observations are made
Variable measured	observed_property	Water Level	What is being measured
Sensor / method	sensor	Pressure Transducer	How the observations are made
Unit	unit	Meter (m)	Unit of the observation values
Processing level	processing_level	Raw (0)	Level of processing/QC
Observation type	observation_type	Field Observation	Type of observation
Result type	result_type	Timeseries	Structure/type of the results
Sampled medium	sampled_medium	Surface Water	Medium being sampled
Missing-data value	no_data_value	-9999	Value representing missing data
Aggregation	aggregation_statistic	Instantaneous	How values are statistically represented
Aggregation interval	time_aggregation_interval	1	Time interval represented by each value
Aggregation interval unit	time_aggregation_interval_unit	minutes	Unit for the aggregation interval
Expected data frequency	intended_time_spacing	15	Expected spacing between observations
Frequency unit	intended_time_spacing_unit	minutes	Unit of the expected spacing
Datastream status	status	Ongoing	Whether collection is ongoing/complete
Public/private	is_private	FALSE	Controls access to the data
Visible/hidden	is_visible	TRUE	Controls whether the datastream is displayed

Learn more: [HydroServer Data Model](#) | [OGC SensorThings API](#)



[Open Geospatial Consortium](#)



[OGC SensorThings API](#)



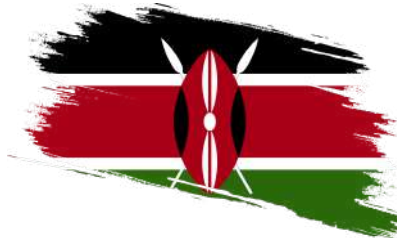
[HydroServer Data Model](#)



Interactive Exercise: Create the Data Model for your Monitoring Station

(approx. 35 minutes)





Instructions



1. Work with your country team.
2. You will receive cards with HydroServer monitoring site properties (e.g., name, site_type, latitude).
3. Find the corresponding information in the metadata for your country's monitoring site. Use the monitoring site from the sample dataset you shared.
4. Write your station's value on the paper provided.
5. Place your value next to the corresponding HydroServer property on the wall.
6. Identify any missing metadata or information that does not clearly map to a HydroServer property.
7. Review the final mapping together.



Goal: Build your monitoring site using the HydroServer data model before creating it in HydroServer.



You can access your sample data, which you shared with us before the workshop, using the links below:



[Kenya: Access Sample Data](#)



Rwanda: [Access Sample Data](#)

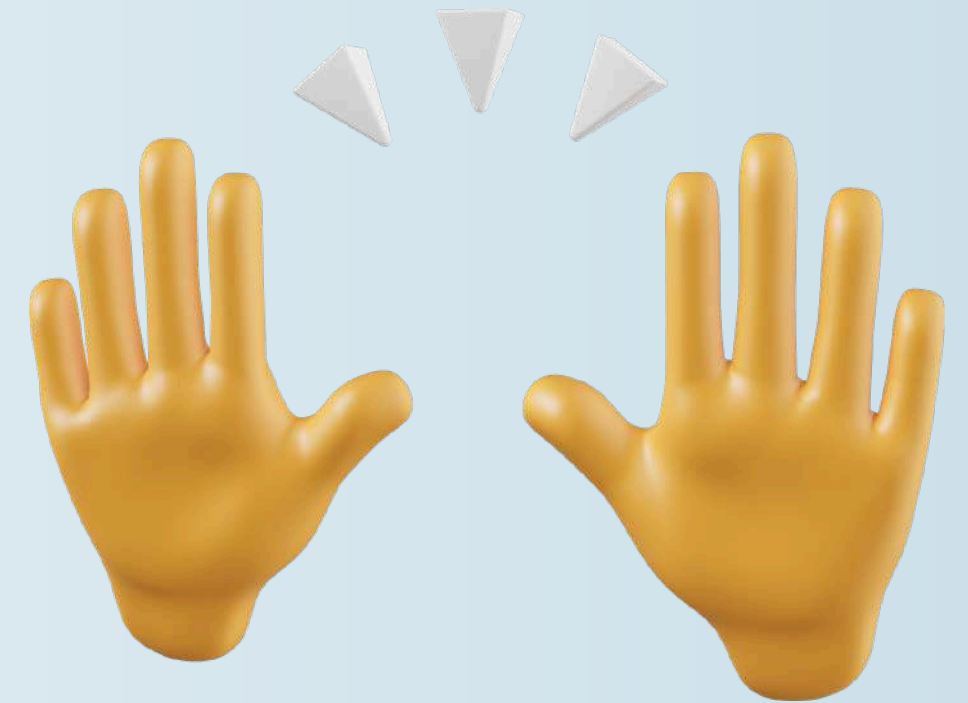


Uganda: [Access Sample Data](#)



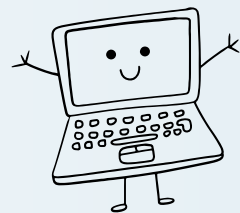
Ethiopia: [Access Sample Data](#)

Good job! 🎉 😎



Now that you understand how HydroServer works and the reasoning behind its design,

you're ready to start working with real-world use cases!



See you in the next session!

Now, take a break!

Please be back at 11:00

